TIFFINLY

PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF

BACHELOR OF COMPUTER APPLICATIONS

To

MARIAN COLLEGE KUTTIKKANAM AUTONOMOUS

Affiliated to

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

By

MEKHNA ALPHONS JOBY

(Reg.No:23UBC240)

Guided By

Ms. ROSAMMA K S

DEPARTMENT OF COMPUTER APPLICATIONS

MARIAN COLLEGE KUTTIKKANAM (AUTONOMOUS)

PEERMADE – 685531

OCTOBER, 2025

DECLARATION

I, **MEKHNA ALPHONS JOBY [Reg.no 23UBC240],** certify that the Mini project report entitled **“TIFFINLY”** is an authentic work carried out by me at Marian College Kuttikkanam (Autonomous). The matter embodied in this project work has not been submitted elsewhere for the award of any degree or diploma to the best of our knowledge and belief.

Signature of the Student:

MEKHNA ALPHONS JOBY

Date:

BONAFIDE CERTIFICATE

This is to certify that this project work entitled **"TIFFINLY"** is a bonafide record of work done by **Ms. MEKHNA ALPHONS JOBY [Reg.no 23UBC240]** at Marian College Kuttikkanam (Autonomous) in partial fulfilment for the award of the **Degree of Bachelor of Computer Applications** of **Mahatma Gandhi University, Kottayam.**

This work has not been submitted elsewhere for the award of any degree to the best of our knowledge.

Head of the Department	Internal Guide
Dr Rajimol A	**Ms. Rosamma K S**
Dept. of Computer Applications	Dept. of Computer Applications
Marian College Kuttikkanam	Marian College Kuttikkanam
Peermade – 685531	Peermade - 685531

Submitted for the Viva-Voice Examination held on

DEPARTMENT SEAL

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

"Gratitude is a feeling which is more eloquent than words, more silent than silence." In undertaking this project, I needed the direction, assistance and cooperation of various individuals and organizations, which is received in abundance with grace of God, without their unconstrained support, the project could not have been completed. If words are considered as the symbol of approval and token of acknowledgement, then let the following words play the heralding role of expressing my gratitude.

I wish to acknowledge our sincere gratitude to our Manager, **Rev.Fr. Boby Alex Mannamplackal** and **Dr. Ajimon George**, Principal **Marian College Kuttikkanam (Autonomous),** for all their efforts and administration in educating me in their premier institution.

I extend my gratitude to **Dr. Rajimol A**, Head of the Department of Computer Application, who is a constant source of inspiration and whose advice helped me to complete this project successfully. I express my deep sense of gratitude to our internal project guide,
Ms. Rosamma K S, for her profound guidance for the successful completion of this project.
With great enthusiasm I express my gratitude to all the faculty members of Department of Computer Applications for their timely help and support.

Finally, I express my deep appreciation to all my friends and family members for the moral support and encouragement they have given to complete this project successfully.

Mekhna Alphons Joby

ABSTRACT

ABSTRACT

The project named **“Tiffinly”** is a web-based meal subscription and delivery management platform that connects customers with affordable food through flexible subscription plans. Designed for students and working professionals, the system allows customers to browse meal options, customize preferences, manage subscriptions, schedule deliveries, and make secure payments.

The platform is role-based:

- **Customers** can browse plans, subscribe, customize meals, track orders, and submit feedback.
- **Admins** manage meals, plans, subscriptions, partners, payments, and respond to inquiries.
- **Delivery Partners** accept and update deliveries, view history, and log issues.

Built with **HTML, CSS, JavaScript, PHP, and MySQL**, Tiffinly supports full CRUD operations and real-time updates via AJAX. Core features include subscription management, cart and checkout, feedback, delivery tracking, and payout handling. It provides an end-to-end digital solution for streamlining the subscription-based meal service system.

TABLE OF CONTENTS

INTRODUCTION

1.INTRODUCTION

1.1 ABOUT THE PROJECT

Tiffinly – A Meal Subscription Platform digitizes the process of ordering and delivering healthy meals through subscription models. It is built for students, professionals, and individuals living away from home.

The system offers **Basic and Premium meal plans** with weekly schedules (Weekdays, Extended Week, Full Week). Customers can customize dietary preference (Basic Plan feature) and meals (Premium Plan feature), add plans to cart, set delivery preferences, and pay securely.

Admins oversee the platform's operations including meal listings, subscriptions, delivery partner management, and payments. Delivery partners can log in to accept orders, update statuses, and track their earnings.

1.1.1 THE PURPOSE AND SCOPE

The main purpose of the Tiffinly project is to develop a unified and user-friendly web application that digitizes the process of ordering and delivering meals through subscription plans. It bridges the gap between cloud kitchens and customers by streamlining every step - meal browsing, customization, carting, payment, delivery, and feedback.

This platform is intended to cover the complete subscription cycle – meal browsing, customization, carting, delivery scheduling, payment, tracking, feedback, and administration.

The scope of Tiffinly encompasses a complete food subscription cycle. Key functionalities include:

- **For Customers:**
    - Browse and compare meal plans
    - Select a plan and based on it save preferences and activate a subscription
    - Schedule deliveries and make payments
    - Track order history
    - Submit feedback and raise inquiries

- **For Admins:**
    - Add/edit/delete meals and plans
    - Get an overview of customer accounts and subscriptions
    - Assign deliveries if required and monitor delivery process
    - Analyse feedbacks and responds to inquiries
- **For Delivery Partners:**
    - View unassigned orders
    - Accept and complete deliveries
    - Update delivery status and log issues
    - Analyse performance and earnings.

The platform is designed for scalability and supports role-based access, secure transactions, and efficient data management using PHP, MySQL, HTML, CSS, and JavaScript.

1.2 EXISTING SYSTEM

In the current scenario, individuals rely on local tiffin providers or food delivery services, which often lack customization, transparency, and consistency. Manual processes lead to miscommunication, delayed deliveries, and no clear feedback mechanism. The absence of digital scheduling and weekly meal planning also causes inefficiencies in kitchen operations and meal preparation.

1.3 PROPOSED SYSTEM

The proposed Tiffinly system resolves the limitations of the existing manual and semi-digital services by offering a fully integrated web-based solution. It allows customers to choose meal plans, customize preferences, handle payments and track deliveries through a centralized dashboard. Admins and delivery partners are provided with tools to manage operations efficiently from their dashboards.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 PROBLEM DEFINITION

In the existing meal delivery ecosystem, customers face challenges in accessing healthy meals on a consistent basis. Many rely on unstructured local services that offer limited customization, inconsistent quality, and no systematic scheduling or tracking. Manual processes dominate both user requests and meal preparation, leading to missed deliveries, lack of transparency, and absence of real-time feedback.

There is also no provision for customizing plans based on dietary preferences. Administrators and delivery partners work in silos, with little to no automation in monitoring or communication. This results in operational inefficiencies and dissatisfied customers.

2.1.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed Tiffinly platform aims to overcome the limitations of the existing system through a modern, digital interface that connects customers, admins, and delivery agents on a unified platform.

Advantages include:

- **Customization:** Customers can select plan based on dietary preference
- **Time Efficiency:** Customers schedule entire subscription duration deliveries in one go, reducing daily ordering stress.
- **Digital Management:** Subscription management, meal selections, payments, and order tracking are fully digitized.
- **Transparency:** Meal plans, prices, delivery status, and order history are clearly accessible.
- **Feedback Loop:** Built-in rating and review system ensures continuous improvement.
- **Role Separation:** Customers, admins, and delivery partners access the system through dedicated interfaces.

2.2 FEASIBILITY ANALYSIS

Feasibility study is a test of a system proposal according to its workability, ability to meet user needs and effective use of resources. The objective of feasibility is not to solve the problem but to acquire a sense of its scope. The main aim of the feasibility study is to test the technical, social and economic feasibility of the system.

The feasibility study can be classified into the following categories:

- Operational Feasibility
- Technical Feasibility
- Economic Feasibility

2.2.1 OPERATIONAL FEASIBILITY

The proposed system is highly user-friendly, with intuitive interfaces for all user roles. End-users (customers) benefit from a smooth ordering experience, while admins and delivery partners are provided with tools to streamline their workflow. Workload is reduced through digital processes.

2.2.2 TECHNICAL FEASIBILITY

Technical feasibility deals with hardware as well as software requirements and to what extend it can support the proposed system. The system uses standard and widely-supported technologies:

- Frontend: HTML, CSS, JavaScript
- Backend: PHP
- Database: MySQL

All components are supported by XAMPP, making it easy to develop, test, and deploy the system on any standard computing environment without expensive infrastructure.

The hardware required is a computer or laptop, and the software required is XAMPP and a web browser. If the necessary requirements are made available with the system, then the proposed system is said to be technically feasible.

2.2.3 ECONOMIC FEASIBILITY

Economic feasibility is an important factor. The proposed system introduces digitization and validation checks, which will significantly reduce operational costs by saving time, minimizing errors, and decreasing dependency on manual processing. Since the development is done using open-source technologies and local resources (e.g., laptops running XAMPP), the financial cost is minimal. Once implemented, the digitized system significantly reduces human resource dependency and operational costs. The long-term savings and benefits—including improved efficiency, greater reliability, and higher customer satisfaction—ensure a high return on investment, making this a cost-effective and economically sound solution. Thus, the benefits acquired out of the system are sufficient enough for the project to be undertaken. So, the proposed system is economically feasible.

2.3 RECOMMENDED IMPLEMENTATIONS

Two principal sources of data are:

- Written documents
- Data from the persons, who are involved in the operation of the system under study.

For successful implementation, the following methods were used during the research and planning phases:

- Questionnaires
- Personal Interviews
- Observations

Questionnaires

Questionnaires are one of the best ways to gather opinions and preferences from end customers. In this project, questionnaires were shared with students, working professionals, and homemakers who typically use tiffin services. The aim was to understand their food habits, delivery expectations, and level of satisfaction with current systems. Most customers preferred a weekly meal planning system with customization options. The data collected helped shape the user experience features in Tiffinly.

Personal Interviews

Personal interviews are a direct and reliable method to gather detailed information. This method was the primary source of fact-finding for the Tiffinly project. Discussions were held with local tiffin service providers, delivery personnel, and students living away from home. They shared insights about issues in the current manual system, such as miscommunication, forgotten orders, and irregular deliveries. Their suggestions for features like scheduled delivery slots, customizable plans, and order tracking were carefully considered and included in the system.

Observations

Observing how local food delivery services operate on a daily basis provided valuable insights. Meals were usually prepared based on calls or fixed routines, with handwritten notes used to track orders. There was no proper system to manage customer preferences, timing, or delivery updates. Watching these manual processes in action helped us understand the gaps and design a digital platform that simplifies operations for admins, enhances convenience for customers, and streamlines delivery for partners. The structure of these observations strongly influenced the database and workflow design of the Tiffinly system.

SOFTWARE REQUIREMENT SPECIFICATIONS

3. SOFTWARE REQUIREMENT SPECIFICATION

Requirements specification is the starting step for the development activities. It is currently one of the weak areas of software engineering. During requirement specification, the goal is to produce a document of the client's requirements. This document forms the basis of development and software validation. The basic reason for the difficulty in software requirements specification comes from the fact that there are three interested parties- the client, the end customers and the software developer.

3.1 PURPOSE

The origin of most software systems is in the need of a client, who either wants to automate an existing manual system or desires a new software system. The software system itself is created by the developer. Finally, the completed system will be used by the end customers. Thus, there are three major parties interested in a new system: the client, the customers and the developer. A basic purpose of software requirements specification is to bridge the communication gap. SRS is the medium through which the client and user needs are accurately specified. Indeed, SRS forms the basis of software development. A good SRS should satisfy all the parties, something very hard to achieve, and involves trade-offs and persuasion.

Another important purpose of developing an SRS is helping the clients understand their own needs. Advantages are:

- An SRS establishes the basis for agreement between the client and the supplier on what the software product will do
- An SRS provides a reference for validation of the final product
- A high-quality SRS is a prerequisite to high-quality software.
- A high-quality SRS reduces the development cost.

3.2 SCOPE

3.2.1 SYSTEM STATEMENT OF SCOPE

The Tiffinly Meal Subscription Platform is developed to provide a complete digital solution for subscribing to meal plans. It offers a user-friendly web interface where customers can register, browse meal options, customize plans, and subscribe to affordable food packages.

The system supports real-time order scheduling and delivery tracking. Customers can view upcoming and past orders, give feedback, and securely manage payments. Admins manage the entire ecosystem including meals, customers, delivery partners, subscriptions, and delivery coordination. Delivery partners interact through their dedicated dashboard to accept orders and update delivery statuses.

All roles access the system through secure, role-based logins. The platform is built with scalability in mind and supports both local and campus-wide use cases.

3.3 TECHNICAL OVERVIEW

3.3.1 USER CHARACTERISTICS

The Tiffinly platform supports three user types:

1. **Customer:**
   - Can register and subscribe to meal plans.
   - Can schedule delivery preferences.
   - Can make payments, view history, rate services and raise inquiries.
2. **Admin:**
   - Has full access to all customers and subscription records
   - Manages meal items, subscription plans, customers, and deliveries
   - Analyse feedback and respond to inquiries.
3. **Delivery Partner:**
   - Can view available orders and accept deliveries
   - Update delivery statuses (Out for Delivery, Delivered, Cancelled)
   - Track delivery history
   - Monitor performance and earnings

Each role sees only its relevant dashboard; features and permissions are hidden from non-authorized customers.

3.4 FUNCTIONAL REQUIREMENTS

The functional requirements of this website are as follows:

1. **User Management:**

- User Registration Process:

  Based on the selected role,
    - Customers register with their name, email, phone number, password and a security question with answer.
    - Delivery partners register with their name, email, phone number, password, a security question with answer, vehicle type, vehicle number, license number, license proof, Aadhaar number and availability.
- User Authentication:
    - Users log in to their respective dashboards by entering their registered email and password.
    - The system verifies credentials against stored data to allow access.
    - Users can securely log out, ending their active sessions to prevent unauthorized access.

2. **Meal Plan Management:**
    - Customers can view available meal plans.
    - Meal details (day-wise) for a week are displayed for available plans.
    - Customers can choose dietary preferences (basic plan feature) or customize meals (premium plan feature) and set delivery preferences for the subscription duration based on schedule chosen (Weekdays, Extended Week (including Saturday), Full Week (including Sunday)).

3. **Subscription and Scheduling:**
    - Customers can subscribe to meal plans (basic/premium).
    - Customers select preferred delivery time slots and address for the subscription duration.
    - Admin can overview subscription records.

4. **Order and Delivery Management:**
   - Order records store all delivery, meal, and user details.
   - Delivery partners can accept new orders.
   - Delivery status can be updated (Pending → Out for Delivery → Delivered → Cancelled).
5. **Feedback and Rating System:**
   - After delivery, customers can rate meals, service and platform and leave feedback.
   - Admins can view all feedback to monitor service quality and respond to inquiries.

6. **Admin Dashboard:**

   Admins can:
   - Access user records
   - Monitor subscription plans data
   - Manage Meal plans, categories and items
   - Analyse feedback and monitor delivery statuses
   - Respond to inquiries
   - Manage partner payouts

3.5 NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements of this website are as follows:

1. Usability:
   - Clean, user-friendly interface for all roles (customer, admin, delivery partner).
   - Clear navigation and structured layout for easy meal selection and order tracking.

2. Reliability:
   - The system must ensure high reliability in its core functions, such as processing user subscriptions, managing meal selections, and assigning deliveries.
   - It should guarantee consistent and accurate data processing for all stakeholders - customers, delivery partners, and administrators - aiming for correct performance in over 95% of daily operations.

3. Availability:

- The system is designed to be available 24/7 to all registered users (customers, delivery partners, and admins).
- Being a web-based application, it can be accessed from any location with an internet connection, allowing for continuous order management, tracking, and administration.

4. Security:

- User passwords and sensitive data will be encrypted.
- Role-based access control will prevent unauthorized access.

5. Performance:

- The system must ensure quick response times for critical user actions like loading meal plans, customizing orders, and real-time delivery tracking.
- Backend processes and database queries are optimized to efficiently handle a high volume of daily transactions, ensuring the platform remains responsive for all users even during peak meal times.

6. Scalability:

- The architecture is designed to support a growing number of active subscriptions and simultaneous delivery operations.
- The project's modular structure (evident in the separate admin, delivery, and user directories) allows for new features or user roles to be added with minimal impact on existing functionality.

7. Maintainability:

- The code is organized into distinct functional modules and role-specific directories, making it easier for developers to locate code, fix bugs, and implement updates.
- A clear separation between the application logic (PHP files), database configuration (config folder), and user interface elements (css, js folders) simplifies routine maintenance and future development.

3.6 STATED REQUIREMENTS

3.6.1 GENERAL REQUIREMENTS

The Tiffinly platform consists of three main user modules: Customer, Admin, and Delivery Partner.

1. Login

- Only registered customers, delivery partners and admins can log in to the system to avail the services.
- The registered customers, delivery partners and admins use their email ID and password to log in.
- The password must contain an uppercase and lowercase alphabetic characters, numbers, and special characters with a minimum requirement of 8 characters.
- Admins and delivery partners will be redirected to their respective panel when logging in with the predefined email ID and password using the same login form.
- Customers will be directed to the user dashboard of the site when the login is successful.

2. Sign Up

- New customers and delivery partners need to register in order to use the platform's services.
- This includes several fields:

  - Full Name
  - Email Address
  - Phone Number
  - Password
  - Confirm Password
  - Security question with answer
    - Vehicle Type, Vehicle Number, License Number, License Proof, Aadhaar Number and Availability (Additional fields to be filled up if registering as a delivery partner)

3. **Admin Panel**

   The admin can:

   - Access user records
   - Monitor subscription plans data
   - Manage Meal plans, categories and items
   - Analyse feedback and monitor delivery statuses
   - Respond to inquiries
   - Manage partner payouts

4. **Explore Meal Plans**
   - Customers can view available meal plans.
   - Meal details (day-wise) for a week are displayed for available plans.
   - Customers can choose dietary preferences (basic plan feature) or customize meals (premium plan feature) and set delivery preferences for the subscription duration based on schedule chosen (Weekdays, Extended Week (including Saturday), Full Week (including Sunday)).

5. **Customize & Purchase Subscription**
   - Customers can select preferred delivery time slots and addresses and save it.
   - Subscriptions are added to cart and paid online.

6. **Order & Delivery Management**
   - Customers can monitor order status every day.
   - Delivery partners can accept orders and update delivery status.

7. **Rating & Feedback**
   - Customers rate meals, services and platform and give feedbacks.
   - Admins can view reviews for performance monitoring.

3.6.2 INPUTS

The Tiffinly system takes the following inputs to provide its services:

• **User Registration:**
– Full Name
– Email Address
– Phone Number
– Password, Confirm Password
– Security Question with Answer
– Address
– Dietary Preference (veg/non-veg)
– For Delivery Partners:
• Vehicle Type, Vehicle Number, License Number, License Proof
• Aadhaar Number
• Availability

• **Login:**
– Registered Email Address
– Password

• **Meal Plan Discovery & Customization:**
– Plan Type (Basic, Premium)
– Schedule Selected (Weekdays, Saturday, Sunday)
– Meal Selection (for Premium)
– Dietary Preference (for Basic)
– Add to Cart, Compare Plans

• **Subscription & Checkout:**
– Delivery Address
– Preferred Time Slot
– Phone Number
– Payment Details

• **Order & Delivery Updates:**
– Order ID
– Meal Type and Selected Meal
– Delivery Status (Pending / Out for Delivery / Delivered)
– Delivery Partner Assignment

• **Feedback & Support:**

– Rating (1–5) for service, meals, platform

– Comment

– Inquiry/Support Request

• **Admin & Partner Management:**

– User/Partner Profile Data

– Menu/Meal Details

– Subscription and Delivery Data

3.6.3 PROCESSING

The system will perform the following key processing tasks:

• **Data Validation:**

– Validate email format, contact number, password complexity

– Ensure required fields are not empty

– Validate address and payment details

• **Meal Planning & Customization:**

– Users select meals based on plan type and delivery schedule

– Customization for premium plans

– Dietary preference handling

• **Subscription & Cart Management:**

– Store and retrieve user subscription data

– Dynamic cost calculation

– Add to cart, compare plans

• **Payment Processing:**

– Secure payment gateway integration

– Store payment history

– Generate payment confirmation

• Delivery Scheduling & Management:

– Delivery partners view and accept orders

– Update delivery status

– Log delivery issues

• Order Tracking & History:

– Real-time tracking for users and partners

– Store delivery history

• Feedback & Support Management:

– Capture and store user feedback

– Admins review feedback and respond to inquiries

• Admin Operations:

– Manage users, partners, plans, menu, and deliveries

– Generate reports and monitor platform activity

3.6.4 OUTPUTS

The system produces the following outputs:

• Meal Plan Summary:

– Selected meals and delivery schedule

– Customization details

• Order & Subscription Details:

– Order ID, user info, meal selections, delivery time

– Subscription summary

• Order and Payment Confirmation:

– Order details

– Payment confirmation and receipt

• Delivery Tracking:

– Real-time order status for users, partners, and admins

– Delivery logs

• **Feedback Reports:**
– Ratings and comments per meal/order
– Inquiry responses

• **Admin Reports:**
– Summary of subscriptions, user activity, delivery logs
– Earnings and payment reports
– Menu/popular meal analytics

3.7 EXTERNAL INTERFACE REQUIREMENTS

3.7.1 USER INTERFACES

All user interfaces will be GUI interfaces, designed to be clean, accessible, and fast offering high functionality. The interfaces will have a pleasing appearance and intuitive design.

- **Design and Appearance**:
  - The interface will use suitable design elements and pleasing colours to create a comfortable and attractive environment for customers.
  - Consistent design themes will be maintained across all pages.
- **Usability Components**:
  - Textboxes, dropdowns, calendar pickers, sliders and buttons will be used to facilitate easy data entry.
  - Clear labels and instructions will be provided for all input fields.
  - Intuitive navigation elements will help customers move seamlessly through the platform.

3.7.2 HARDWARE INTERFACES

The system needs a computer or any other smart devices with network availability to access the web application. No other external hardware is required.

Hardware Specification

- **Processor**: Intel Pentium or higher
- **RAM**: 256 MB or higher
- **Hard Disk Drive**: 100 MB required on disk
- **Keyboard**: Standard QWERTY keyboard

Implementation Specification

- **Operating System**: Windows OS

3.7.3 SOFTWARE INTERFACES

Software Specification

- **Operating System**: Windows 11 or equivalent
- **Browser**: Chrome, Firefox, Edge (latest versions)
- **Web Server**: XAMPP with Apache and MySQL
- **Programming Language**: PHP
- **Database**: MySQL
- **Frontend**: HTML, CSS, JavaScript

SYSTEM DESIGN

4. SYSTEM DESIGN

The design phase aims to develop a solution for the problems identified during the analysis phase. This phase marks the transition from understanding the problem to creating a solution. System design details the required features and operations, including screen layouts, business rules, process diagrams, pseudocode, and other relevant documentation.

During this phase, the overall structure and specifics of the software are defined. This includes determining the number of tiers needed for the architecture, designing inputs and outputs, and establishing database and data structure designs. Proper analysis and design are critical to the development cycle since errors in this phase can be costly to fix later. Therefore, careful attention is given during the design phase.

The logical framework of the product and its physical attributes are outlined during this stage. The operating environment is set up, and key resources are identified. Any element that requires user input or approval must be documented and reviewed by the user. The physical aspects of the system are specified, and a detailed design is prepared.

Subsystems identified during design are used to create a detailed system structure. Each subsystem is divided into one or more design units or modules, and detailed logic specifications are created for each module. The module logic is typically described in a high-level design language, which is independent of the final implementation language.

A good design should consider:

- **Promptness**: The design should be straightforward and clear, guiding customers to their desired outcomes intuitively.
- **Memory Load**: Research shows that customers can retain about six words in their short-term memory. The number of choices presented to customers should ideally be four or fewer to prevent confusion and forgetfulness.
- **Service Reachability**: Customers dislike going through many steps to access a service. More than five steps can cause impatience, so minimizing the number of steps helps reduce frustration.

- **Navigation**: Customers should easily navigate back and forth between different steps, allowing them to access various parts of the dialog seamlessly.
- **Phonetic Similarity**: Avoid choices with similar pronunciations to reduce confusion and ensure customers can clearly distinguish between options.
- **Error Handling**: Implementing graceful error handling helps decrease dependency on operators by managing mistakes effectively.
- **User Updates**: Keep customers informed about the ongoing process to maintain their engagement and understanding.

For the general design, one or more potential designs are proposed and broadly sketched. These alternatives are then presented to the customers, who choose the design that best suits their requirements while staying within project constraints.

The detailed design stage specifies the user interface, database, programs, hardware, and training and system documentation. Several structured techniques are used during the design phase. To design the software components, the designer transforms the automated processes in the physical data flow diagram into a program structure chart, which decomposes software processes into detailed modules and shows control paths between modules.

4.1 DESIGN METHODOLOGY

4.1.1 INPUT DESIGN

Input design focuses on converting user-oriented inputs into a format recognizable by the computer. Collecting input data is one of the costliest parts of the system in terms of equipment, time, and user involvement. The goal of input design is to make data entry as simple, logical, and error-free as possible.

Input design serves as the bridge between the information system and its customers, transforming transaction data into a form suitable for processing. This process can involve reading data from printed documents or directly keying data into the system. Effective input design controls the amount of input required, minimizes errors, avoids delays and extra steps, and keeps the process straightforward.

System analysis determines the following input design details:

- **What data to input**
- **What medium to use**
- **How the data is arranged and coded**
- **Data items and transactions requiring validation to detect errors**

Activities involved in input design include:

- **Data Recording**: Collecting data for input.
- **Data Verification**: Ensuring the accuracy of the collected data.
- **Data Conversion**: Transforming data into the required format.
- **Data Validation**: Checking data for errors.
- **Data Correction**: Fixing any identified errors.

4.1.1 OUTPUT DESIGN

Output design involves creating necessary outputs tailored to meet the requirements of various customers. It is essential to approach the design of computer outputs in a well-considered manner. Outputs refer to any information generated by the information system, whether in printed or displayed form. Analysts design computer outputs by identifying specific outputs required to fulfil system requirements.

Computers serve as crucial sources of information for customers. Efficient and thoughtful output design enhances the system's interaction with customers and supports decision-making processes. When designing outputs, system analysts must achieve the following objectives:

- Determine the information to be presented
- Decide whether to display, print, or verbally communicate the information, and select the appropriate output medium
- Format the information in an easily understandable manner

Output design is critical to the success of any system as it bridges the gap between the user and the system's operations. Effective output design includes specifications and procedures for presenting data clearly to customers. Customers should never be left uncertain about system activities, as appropriate error messages and acknowledgment messages are provided.

4.1.3 CODE DESIGN

The coding phase transforms the detailed software design into a programming language. It translates the software's detailed design representation into executable code. Code design aims to minimize the lines of code used while modularizing the implementation. Modules hide complexity by encapsulating executable statements under named functions or procedures. Effective information hiding enhances program understanding at higher abstraction levels. Module names should accurately describe their actions to avoid confusion.

In this software, a modularized approach is employed with different functions created for various operations, each named to reflect its action.

4.1.4 DATABASE DESIGN

Database design identifies relevant data relationships and defines tables using standard methods. Each table's attributes are carefully defined to optimize database performance, ensure data integrity, minimize redundancy, and enhance security.

A database system is a computer representation of an information system designed to handle integrated data efficiently. It minimizes redundancy to provide quick, flexible, and cost-effective information access. Database design considers several specific objectives:

- Controlled redundancy
- User-friendly interface
- Data independence
- Cost-effective data retrieval
- Accuracy and integrity
- Failure recovery
- Privacy and security
- Performance optimization

Database design involves creating multiple views of data, including logical and physical views. The logical view represents data independently of its storage, focusing on how customers and programmers interact with it. The physical view describes how data are stored and accessed in physical storage.

Each table in the database typically includes a primary key, a unique column (or combination of columns) that uniquely identifies each record. Primary keys ensure data integrity by enforcing uniqueness and cannot contain null values.

Normalization is employed to organize database data, minimizing redundancy and anomalies during data insertion, update, and deletion operations.

4.2 SYSTEM ARCHITECTURE AND PROCESS FLOW

UML DIAGRAMS

4.2.1 USE CASES

REGISTRATION

Use Case Id:	TF_UC_01
Use Case Name:	Registration
Created by:	Mekhna
Date Created:	28-06-2025
Description:	Allows new customers and partners to register for an account on the platform.
Primary actor:	Customer, Delivery Partner
Secondary actor:	None
Precondition:	User navigates to the registration page
Postcondition:	User account is successfully created.
Main flow:	1. User navigates to the registration page. 2. User provides required details: full name, email, phone no, password, confirm password, a security question and an answer. For delivery partners registration, additional fields are required: vehicle type, vehicle number, license number, license proof document, Aadhaar number, and availability 3. User submits the registration form. 4. System validates the information and creates the user account. 5. Use case ends.

LOGIN

Use Case Id:	TF_UC_02
Use Case Name:	Login
Created by:	Mekhna

Date Created:	29-06-2025
Description:	This use case enables users to access the system by entering their credentials. There are three user roles: admin, delivery partner and customer. The admin logs in using their admin email and password, while the customers and delivery partners uses their own email and password. Once logged in, customers can purchase, delivery partners can view available orders and accept deliveries and admins can add or edit items on the website.
Primary actor:	Customer, Admin, Delivery partner
Secondary actor:	None
Precondition:	The user should have a valid account.
Postcondition:	The system displays relevant dashboard.
Main flow:	1. The user goes to the login page. 2. The user inputs their registered email and password. 3. The user submits the login form. 4. The system checks the user's credentials. 5. If the credentials are correct, the system grants the user access. 6. The use case concludes.

MEAL PLAN DISCOVERY

Use Case Id:	TF_UC_03
Use Case Name:	Meal Plan Discovery
Created by:	Mekhna
Date Created:	03-07-2024
Description:	This use case allows users to explore available meal plans, view details, compare options, and customize their selections. Users can add preferred plans to their cart and proceed toward subscription.
Primary actor:	Customer
Secondary actor:	Admin
Precondition:	The user should be logged into their account.

Postcondition:	The system displays detailed information about meal plans and updates the cart when selections are made.
Main flow:	1. The user navigates to the meal plan discovery page. 2. The system displays available meal plans with details. 3. The user selects a meal plan to view or compare. 4. The system retrieves and displays detailed meal plan information. 5. The user adds the plan to the cart. 6. The use case concludes.

ADMIN DASHBOARD

Use Case Id:	TF_UC_04
Use Case Name:	Admin Dashboard
Created by:	Mekhna
Date Created:	04-07-2025
Description:	This use case provides administrators with tools to oversee accounts, manage meal plans, monitor deliveries, and handle user queries.
Primary actor:	Admin
Secondary actor:	None
Precondition:	Admin must log into the system with valid credentials.
Postcondition:	The system reflects updated management actions and ensures smooth operations.
Main flow:	1. The admin logs into the dashboard. 2. The system displays an overview of accounts, meal plans, and deliveries. 3. The admin selects a management function (e.g., update meal plans, meals, pay partners). 4. The system processes updates or queries. 5. The admin reviews feedback or resolves issues. 6. The use case concludes.

ORDER TRACKING & HISTORY

Use Case Id:	TF_UC_05
Use Case Name:	Order Tracking & History
Created by:	Mekhna
Date Created:	04-07-2025
Description:	This use case allows users and delivery partners to track current deliveries and view past order history.
Primary actor:	Customer, Delivery Partner
Secondary actor:	Admin
Precondition:	The user/delivery partner must have active or past orders.
Postcondition:	The system displays tracking updates and history.
Main flow:	1. The user accesses the tracking page. 2. The system retrieves current delivery status. 3. The user views live tracking updates. 4. The user checks past delivery history and details. 5. The use case concludes.

PAYMENT & EARNINGS

Use Case Id:	TF_UC_06
Use Case Name:	Payment & Earnings
Created by:	Mekhna
Date Created:	05-07-2025
Description:	This use case manages secure payment processing for users and tracks earnings for delivery partners.
Primary actor:	Customer, Delivery Partner
Secondary actor:	Admin
Precondition:	The user must have a subscription, and delivery partners must have completed deliveries.

Postcondition:	Payment is securely processed and recorded.
Main flow:	1. The user selects a subscription payment option. 2. The system processes the transaction securely. 3. The user views payment history. 4. Delivery partners view earnings and payment status based on number of meals delivered. 5. The admin monitors transactions. 6. The use case concludes.

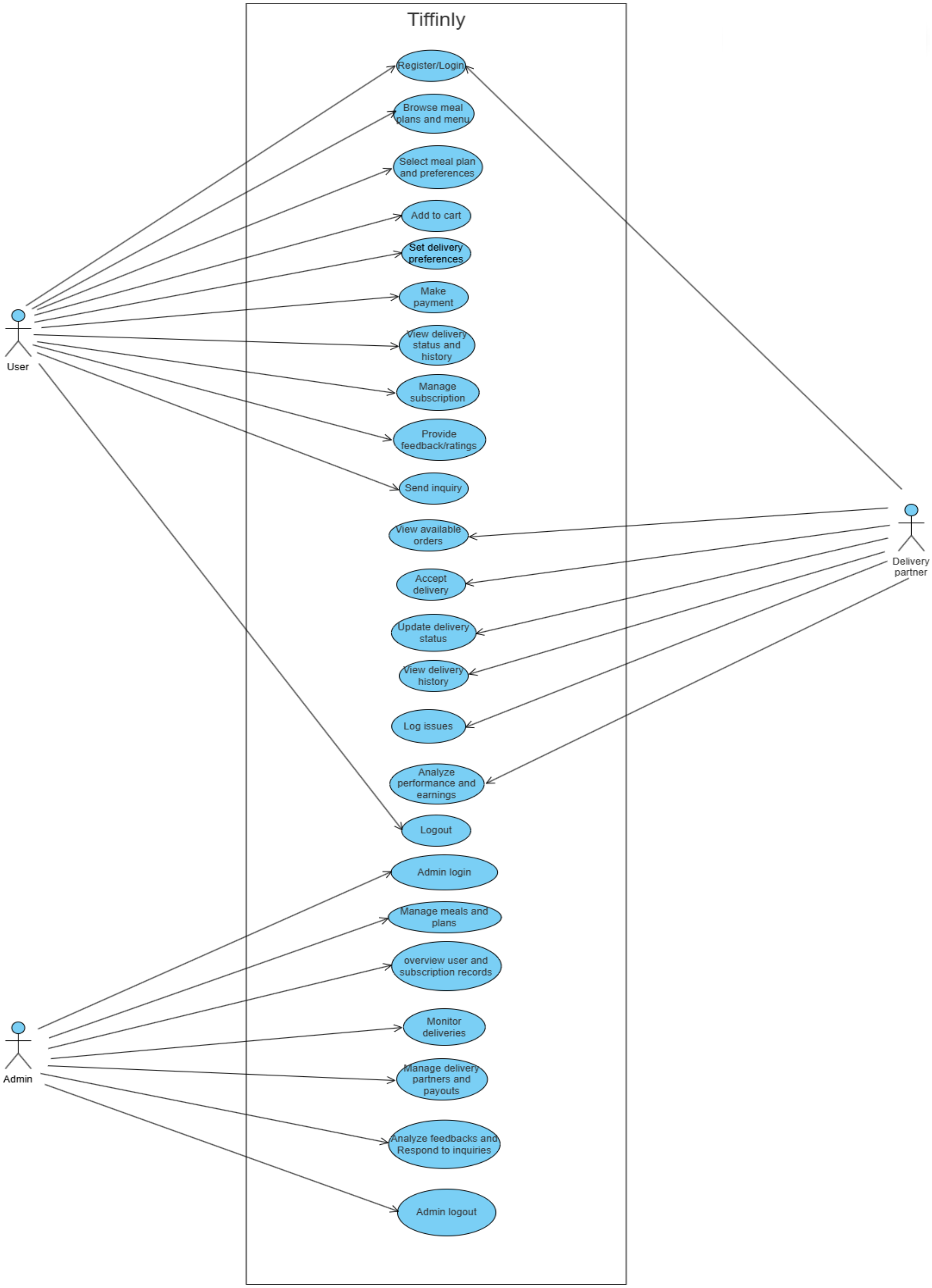
INQUIRY MANAGEMENT

Use Case Id:	TF_UC_07
Use Case Name:	Inquiry Management
Created by:	Mekhna
Date Created:	06-07-2025
Description:	This use case allows users to submit inquiries and administrators to respond and track them.
Primary actor:	User
Secondary actor:	Admin
Precondition:	User must have an account.
Postcondition:	The system records and tracks inquiries and responses.
Main flow:	1. The user navigates to the inquiry page. 2. The user submits a question or request. 3. The system records the inquiry. 4. The admin reviews and responds. 5. The system notifies the user. 6. The use case concludes.

LOG OUT

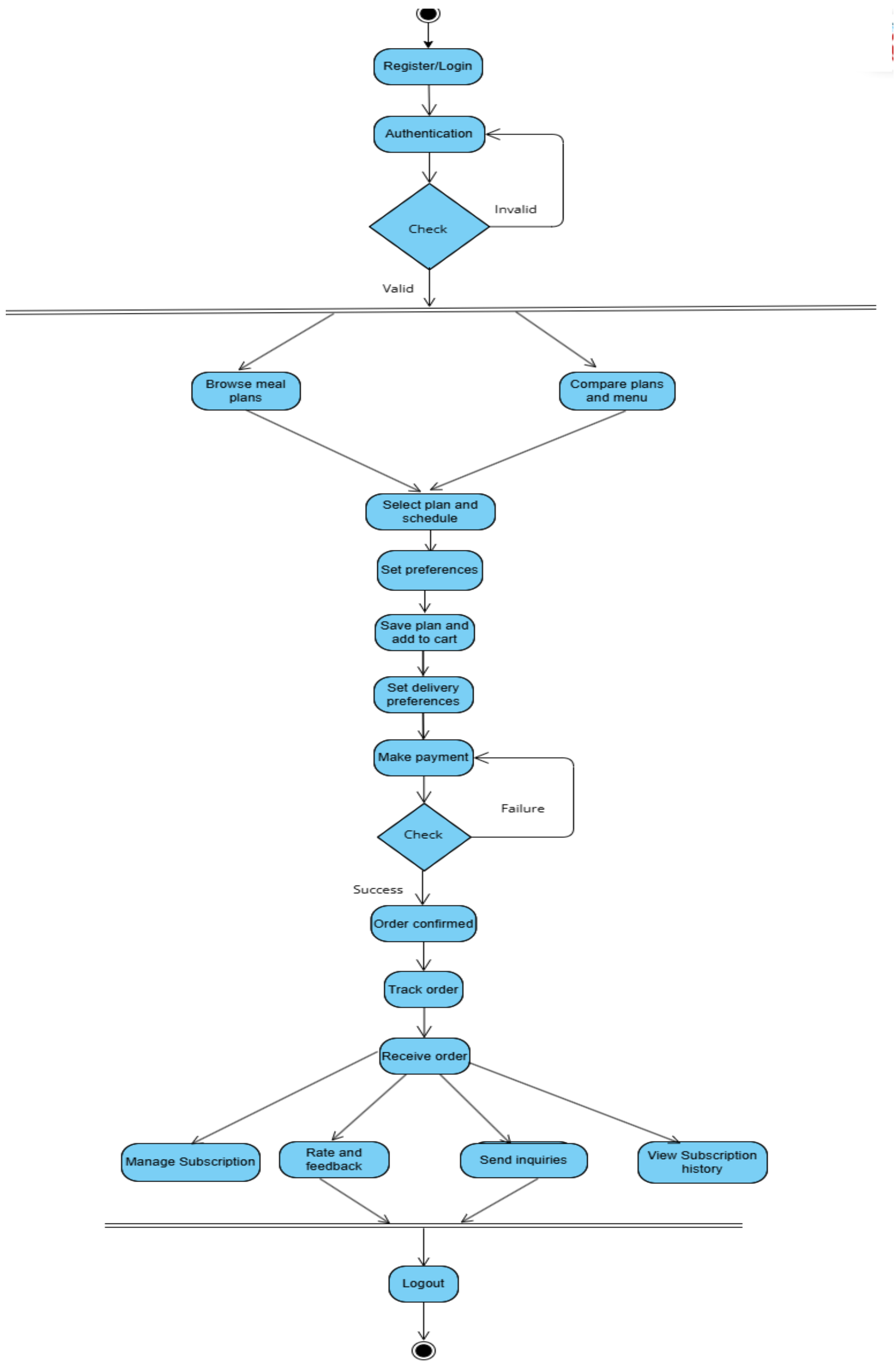
Use Case Id:	TF_UC_10
Use Case Name:	Log Out
Created by:	Mekhna
Date Created:	30-06-2024
Description:	This use case allows customers, partners and admins to log out of their accounts on the platform. Logging out ensures that the session is securely ended, and access is restricted until the user or admin logs in again.
Primary actor:	Customer/Admin/Delivery Partner
Secondary actor:	None
Precondition:	The user or admin must be logged into their account.
Postcondition:	The user or admin must be logged into their account.
Main flow:	1. The user or admin navigates to the log out option. 2. The user or admin selects the option to log out. 3. The system terminates the session of the user or admin. 4. The system redirects the user or admin to the login page. 5. The system confirms that the user, partner or admin has been successfully logged out. 6. Use case ends.

4.2.2 USECASE DIAGRAM

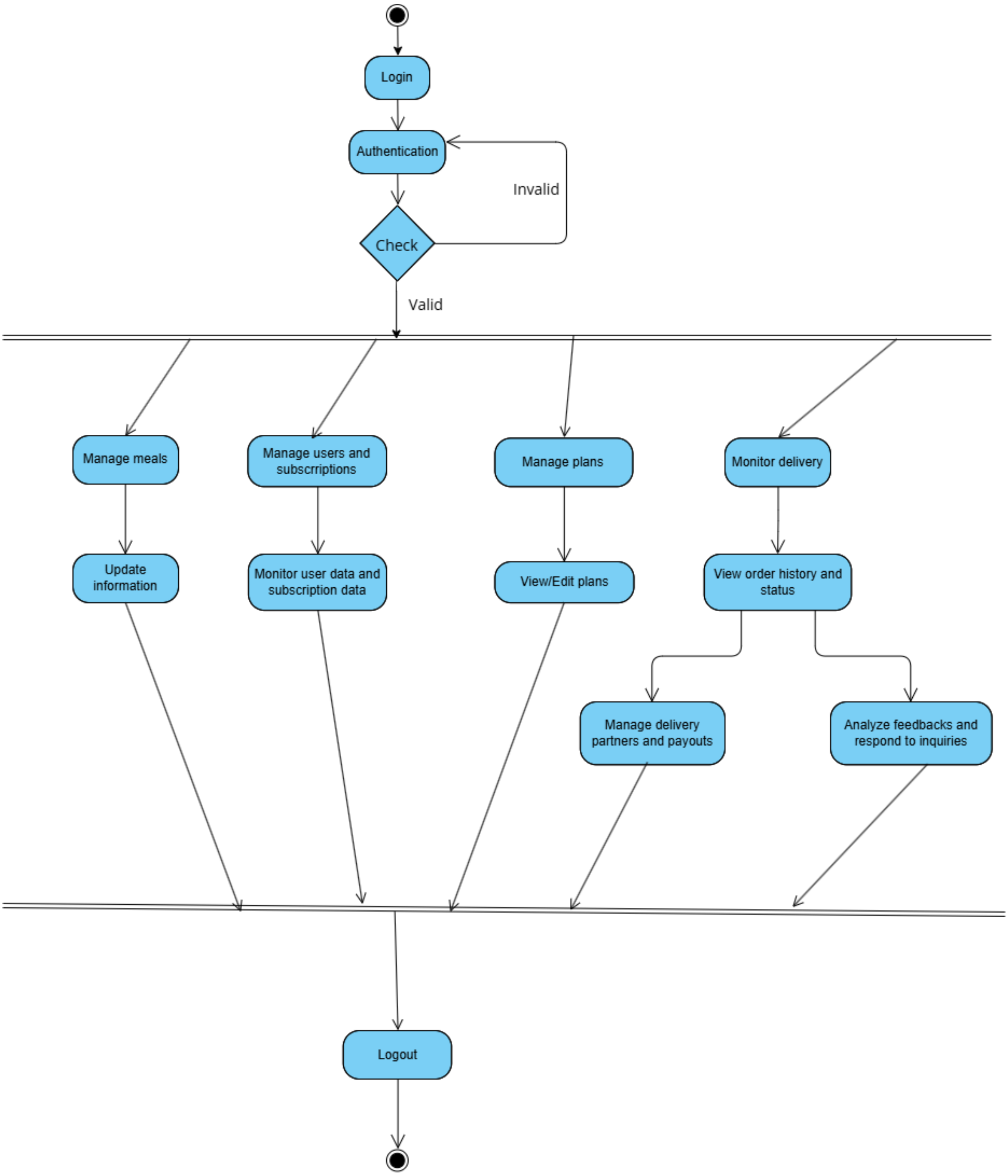


4.2.3 ACTIVITY DIAGRAMS

<u>USER SIDE</u>

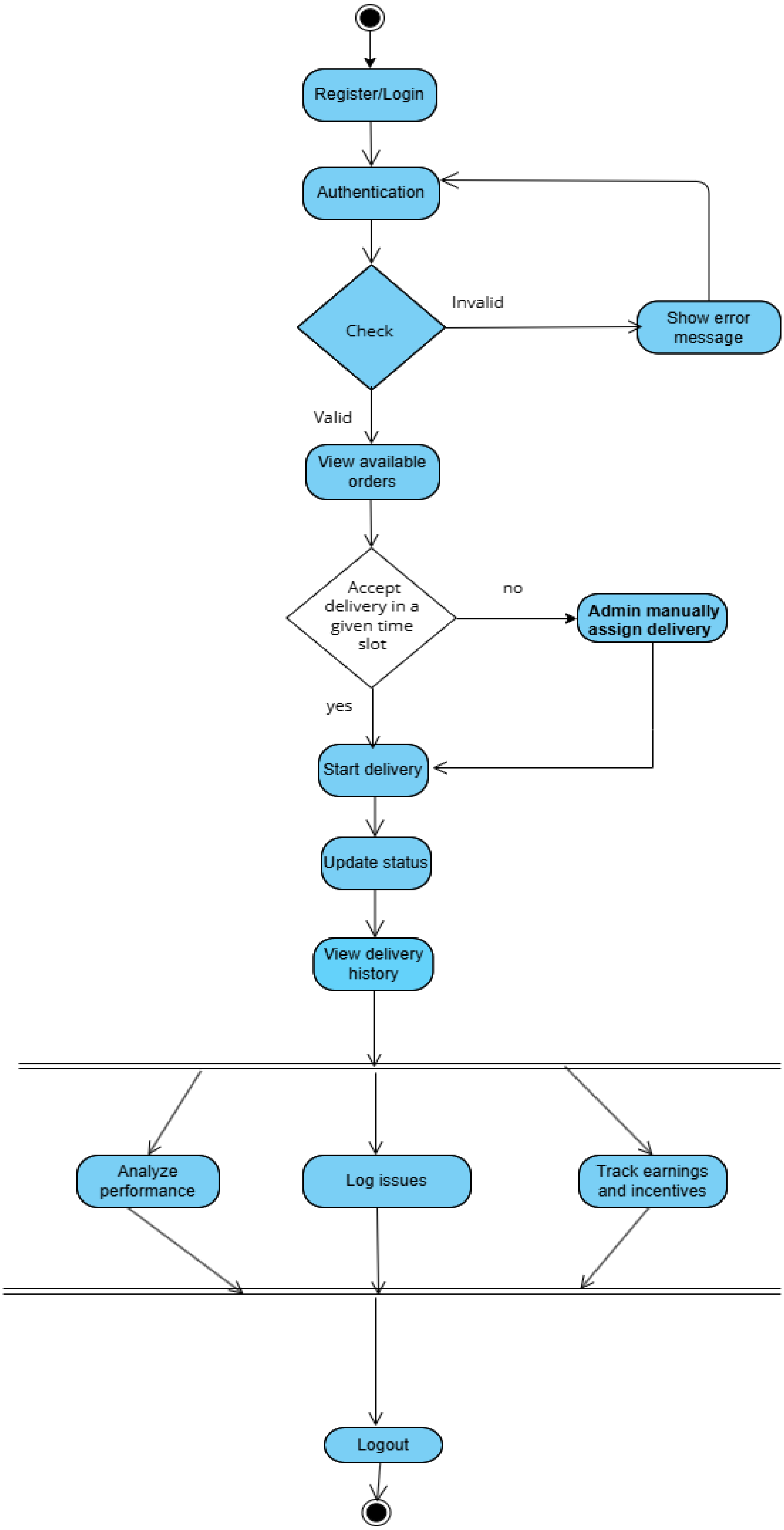


ADMIN SIDE



Login
Authentication
Invalid
Check
Valid
Manage meals
Manage users and subscrriptions
Manage plans
Monitor delivery
Update information
Monitor user data and subscription data
View/Edit plans
View order history and status
Manage delivery partners and payouts
Analyze feedbacks and respond to inquiries
Logout

DELIVERY PARTNER SIDE



4.3 MODULE DETAILS

There are essentially fourteen modules in the Tiffinly website:

1. Login Module

The login module allows registered users (customers, delivery partners, and administrators) to securely access the Tiffinly platform. It verifies user credentials (email and password) to grant access to personalized services such as browsing meal plans, managing subscriptions, tracking deliveries, and accessing admin features.

2. Sign-Up Module

The sign-up module enables new users to create accounts on Tiffinly primarily in two roles-customer and delivery partner. Users provide personal details including name, email, phone number, password, a security question and a security answer to register. Users who wish to join as delivery partners are required to provide additional information like vehicle type, vehicle number, license number, license proof, Aadhaar number and availability. Once registered, users can select meal plans, customize preferences, and manage their subscriptions.

3. Meal Plan Discovery Module

This module allows users to explore available meal plans, view detailed information about meals, compare plans, and customize their selections. Users can add preferred plans to their cart and proceed to subscription.

4. Subscription & Delivery Management Module

This module enables users to subscribe to meal plans, set delivery preferences, track daily deliveries, and provide feedback. It also allows users to view delivery history, manage payment details, and update subscription options.

5. Admin Dashboard Module

The admin dashboard provides administrators with tools to oversee user accounts, manage meal plans, monitor delivery schedules, review feedback, and update website content. Admins can ensure smooth platform operation, maintain data integrity, and resolve user issues.

6. Meal Customization Module

The meal customization module allows users to personalize their meal selections within a chosen plan. Users can select specific meals, and set dietary preferences. This module ensures that users receive meals tailored to their tastes and requirements.

7. Cart & Plan Comparison Module

This module enables users to add meal plans to a cart and compare multiple plans side-by-side. Users can review features, pricing, and meal options before making a subscription decision, improving transparency and user satisfaction.

8. Order Tracking & History Module

The order tracking and history module provides users and delivery partners with real-time updates on current deliveries and access to past delivery records. Users can monitor delivery status, view meal details, and check delivery dates, while partners can track assigned deliveries.

9. Payment & Earnings Module

This module manages payment processing for subscriptions and tracks earnings for delivery partners. Users can view payment history, make secure transactions, and partners can monitor their earnings and payment status.

10. Feedback & Support Module

The feedback and support module allows users to submit feedback on meals and delivery experiences, and access support for any issues. Administrators can review feedback to improve service quality and respond to user queries.

11. Partner Management Module

This module provides tools for administrators and delivery partners to manage partner profiles, assignments, and payments. Partners can update their information, view delivery schedules, and receive payment updates.

12. Menu & Popular Meals Management Module

Administrators use this module to manage the meal menu, update meal details, and highlight popular meals. It ensures the menu remains current and attractive to users.

13. Inquiry Management Module
The inquiry management module allows users to submit questions or requests, and administrators to respond and track inquiries. This helps maintain effective communication and resolve user concerns promptly.

14. Session & Authentication Management Module
This module handles user sessions and authentication checks, ensuring secure access to platform features and protecting user data.

These modules work together to deliver a seamless experience for customers, delivery partners, and administrators on the Tiffinly platform.

4.4 PERFORMANCE CONSIDERATIONS

Hardware Requirements

The system is designed to perform optimally with a minimum of 4GB RAM and is compatible with Windows OS versions and higher.

4.5 SECURITY CONSIDERATIONS

Access Control

- **Authorized Access**: Only customers with valid registered email-id and passwords are allowed to access the Tiffinly platform.
- **Login Security**: The login process includes robust security measures to authenticate customers and prevent unauthorized access.

4.6 TABLE DESIGN

1.Table name: users

Sl. No.	Field Name	Type	Constraint	Description
1	user_id	int(11)	PK	User ID
2	full_name	varchar(100)	NOT NULL	Full name
3	email	varchar(100)	UNIQUE, NOT NULL	User email
4	phone	varchar(15)	UNIQUE, NOT NULL	Phone number
5	password	varchar(255)	NOT NULL	Hashed password
6	role	enum('user', 'admin','delivery')	DEFAULT 'user'	Role type
7	security_question	varchar(255)	NULL	Security question for reset
8	security_answer	varchar(255)	NULL	Security answer
9	created_at	timestamp	DEFAULT current_timestamp()	Account created
10	updated_at	timestamp	DEFAULT current_timestamp() ON UPDATE	Last update

2.Table name: auth_tokens

Sl. No.	Field Name	Type	Constraint	Description
1	id	int	PK, AUTO_INCREMENT, NOT NULL	Unique token record ID
2	user_id	int	FK → users(user_id), NOT NULL, ON DELETE CASCADE	References the user who owns the token

3	selector	varchar(32)	NOT NULL, INDEX	Public identifier for the token (used for quick lookup)
4	validator	varchar(64)	NOT NULL	Secret validator part of the token (stored hashed for security)
5	expires_at	datetime	NOT NULL	Expiration date and time for the token

3.Table name: addresses

Sl. No.	**Field Name**	**Type**	**Constraint**	**Description**
1	address_id	int(11)	PK, NOT NULL	Unique ID for address
2	user_id	int(11)	FK → users(user_id)	Reference to user
3	address_type	enum('home','work')	NOT NULL	Type of address
4	line1	varchar(255)	NOT NULL	Primary address line
5	line2	varchar(255)	NULL	Secondary address line
6	city	varchar(100)	NOT NULL	City
7	state	varchar(100)	NOT NULL	State
8	pincode	varchar(10)	NOT NULL	Postal code
9	landmark	varchar(255)	NULL	Landmark details
10	is_default	tinyint(1)	DEFAULT 1	Marks default address

4.Table name: delivery_partner_details

Sl. No.	**Field Name**	**Type**	**Constraint**	**Description**
1	partner_id	int(11)	PK	Partner ID (linked to user_id)
2	vehicle_type	varchar(50)	NOT NULL	Vehicle type

3	vehicle_number	varchar(20)	NOT NULL	Registration number
4	license_number	varchar(30)	NOT NULL	License no.
5	license_file	varchar(255)	NULL	Uploaded license proof
6	aadhar_number	varchar(20)	NULL	Aadhaar ID
7	availability	enum('Part-time','Full-time')	NOT NULL	Work type

5.Table name: deliveries

Sl. No.	Field Name	Type	Constraint	Description
1	delivery_id	int(11)	PK, NOT NULL	Delivery record ID
2	subscription_id	int(11)	FK → subscriptions (subscription_id)	Linked subscription
3	delivery_date	date	NOT NULL	Scheduled date
4	status	enum('scheduled', ‘out_for_delivery', 'delivered', 'cancelled')	DEFAULT 'scheduled'	Delivery status
5	payment_status	enum('unpaid', 'paid')	DEFAULT 'unpaid'	Payment state
6	delivery_time	time	NULL	Scheduled time

6.Table name: delivery_issues

Sl. No.	Field Name	Type	Constraint	Description
1	issue_id	int(11)	PK	Issue ID
2	assignment_id	int(11)	FK	Related assignment

3	subscription_id	int(11)	FK	Subscription
4	partner_id	int(11)	FK	Delivery partner
5	meal_type	varchar(50)	NULL	Meal affected
6	issue_type	varchar(100)	NULL	Type of issue
7	status	enum('open','resolved')	DEFAULT 'open'	Resolution state
8	description	text	NULL	Details

7.Table name: delivery_assignments

Sl. No.	Field Name	Type	Constraint	Description
1	assignment_id	int(11)	PK	Assignment ID
2	subscription_id	int(11)	FK	Subscription linked
3	delivery_date	date	NOT NULL	Assigned date
4	partner_id	int(11)	FK → delivery_partner_details(partner_id)	Delivery partner
5	assigned_at	timestamp	DEFAULT current_timestamp ()	Assignment timestamp
6	status	enum('pending', 'out_for_delivery', 'delivered', 'cancelled')	DEFAULT 'pending'	Delivery progress
7	meal_type	varchar(20)	NOT NULL	Breakfast/Lunch /Dinner
8	meal_id	int(11)	FK → meals(meal_id)	Assigned meal
9	payment_id	int(11)	FK → partner_payments(payment_id)	Payment record
10	payment_status	enum('unpaid', 'paid')	DEFAULT 'unpaid'	Partner payment

8.Table name: delivery_preferences

Sl. No.	Field Name	Type	Constraint	Description
1	preference_id	int(11)	PK	Preference ID
2	user_id	int(11)	FK	User
3	meal_type	enum('breakfast', 'lunch','dinner')	NOT NULL	Meal choice
4	address_id	int(11)	FK → addresses(address_id)	Delivery address
5	time_slot	varchar(50)	NULL	Preferred time
6	created_at	timestamp	DEFAULT current_timestamp()	Created
7	updated_at	timestamp	DEFAULT current_timestamp() ON UPDATE	Updated

9.Table name: feedback

Sl. No.	Field Name	Type	Constraint	Description
1	feedback_id	int(11)	PK	Unique feedback ID
2	user_id	int(11)	FK → users(user_id)	Feedback given by user
3	feedback_type	enum('meal', 'service','platform')	NOT NULL	Type of feedback
4	rating	tinyint(4)	NOT NULL, CHECK (1–5)	Rating value
5	comments	text	NULL	Additional feedback
6	meal_description	varchar(255)	NULL	Details of meal (if meal feedback)
7	delivery_date	date	NULL	Related delivery date (if service feedback)

8	created_at	timestamp	DEFAULT current_timestamp ()	Created timestamp
9	updated_at	timestamp	DEFAULT current_timestamp () ON UPDATE	Last updated

10.Table name: inquiries

Sl. No.	Field Name	Type	Constraint	Description
1	inquiry_id	int(11)	PK	Inquiry ID
2	user_id	int(11)	FK → users(user_id), NULL allowed	User making inquiry
3	message	text	NOT NULL	Inquiry content
4	inquiry_type	varchar(50)	DEFAULT 'general'	Category (general, technical, etc.)
5	response	text	NULL	Admin response
6	status	enum('pending', 'responded','closed')	DEFAULT 'pending'	Inquiry state
7	created_at	timestamp	DEFAULT current_timestamp()	Logged timestamp

11.Table name: meals

Sl. No.	Field Name	Type	Constraint	Description
1	meal_id	int(11)	PK	Unique meal ID
2	category_id	int(11)	FK → meal_categories(category_id)	Meal category

3	meal_name	varchar(100)	NOT NULL	Name of meal
4	description	text	NULL	Meal description
5	image_url	varchar(255)	NULL	Meal image path
6	is_active	tinyint(1)	DEFAULT 1	Availability flag

12. Table name: meal_categories

Sl. No.	Field Name	Type	Constraint	Description
1	category_id	int(11)	PK	Category ID
2	category_name	varchar(50)	NOT NULL	Category name
3	meal_type	En meal_categories enum('Breakfast','Lunch','Dinner')	NOT NULL	Meal type
4	option_type	enum('veg','non_veg')	NOT NULL	Veg/Non-veg option
5	slot	enum('breakfast','lunch','dinner')	NULL	Delivery slot

13. Table name: meal_plans

Sl. No.	Field Name	Type	Constraint	Description
1	plan_id	int(11)	PK	Plan ID
2	plan_name	varchar(50)	NOT NULL	Plan name
3	description	text	NULL	Plan description

4	plan_type	enum('basic','premium')	NOT NULL	Type of plan
5	is_active	tinyint(1)	DEFAULT 1	Availability
6	base_price	decimal(8,2)	NOT NULL	Base price per day

14. Table name: partner_payments

Sl. No.	Field Name	Type	Constraint	Description
1	payment_id	int(11)	PK	Payment ID
2	partner_id	int(11)	FK → delivery_partner_details(partner_id)	Partner paid
3	subscription_id	int(11)	FK → subscriptions(subscription_id)	Related subscription
4	delivery_date	date	NULL	Delivery date
5	amount	decimal(10,2)	NOT NULL	Payment amount
6	delivery_count	int(11)	DEFAULT 0	Number of deliveries covered
7	payment_method	varchar(50)	NOT NULL	Method (e.g., Razorpay)
8	payment_status	enum('success', 'failed', 'pending')	DEFAULT 'success'	Payment state
9	transaction_ref	varchar(100)	NULL	Transaction reference
10	created_at	timestamp	DEFAULT current_timestamp ()	Created timestamp

15. Table name: payments

Sl. No.	Field Name	Type	Constraint	Description
1	payment_id	int(11)	PK	Payment ID
2	user_id	int(11)	FK → users(user_id)	User making payment
3	subscription_id	int(11)	FK → subscriptions (subscription_id)	Related subscription
4	amount	decimal(10,2)	NOT NULL	Paid amount
5	payment_method	varchar(50)	NOT NULL	Payment method
6	payment_status	enum('pending', 'success','failed')	DEFAULT 'pending'	Status
7	transaction_ref	varchar(100)	NULL	Transaction reference
8	created_at	timestamp	DEFAULT current_timestamp()	Created timestamp

16. Table name: plan_features

Sl. No.	Field Name	Type	Constraint	Description
1	feature_id	int(11)	PK	Feature ID
2	plan_id	int(11)	FK → meal_plans(plan_id)	Linked meal plan
3	feature_text	varchar(255)	NOT NULL	Description of feature
4	sort_order	int(11)	DEFAULT 0	Ordering

17. Table name: plan_images

Sl. No.	Field Name	Type	Constraint	Description
1	image_id	int(11)	PK	Image ID

2	plan_id	int(11)	FK → meal_plans(plan_id)	Linked meal plan
3	image_url	varchar(255)	NOT NULL	Path to image
4	sort_order	int(11)	DEFAULT 0	Ordering

18. Table name: subscription_meals

Sl. No.	Field Name	Type	Constraint	Description
1	id	int(11)	PK	Record ID
2	subscription_id	int(11)	FK → subscriptions(subscription_id)	Linked subscription
3	delivery_date	date	NOT NULL	Date of delivery
4	meal_type	enum('breakfast','lunch','dinner')	NOT NULL	Meal type
5	meal_id	int(11)	FK → meals(meal_id)	Actual meal delivered
6	status	enum('scheduled','cancelled','delivered')	DEFAULT 'scheduled'	Meal delivery status

19. Table name: plan_meals

Sl. No.	Field Name	Type	Constraint	Description
1	plan_meal_id	int(11)	PK	Record ID
2	plan_id	int(11)	FK → meal_plans(plan_id)	Related plan
3	day_of_week	enum('MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY',	NOT NULL	Day of week

		'SATURDAY', 'SUNDAY')		
4	meal_type	enum('Breakfast','Lunch','Dinner')	NOT NULL	Type of meal
5	meal_id	int(11)	FK → meals(meal_id)	Meal linked

20. Table name: plan_schedule_options

Sl. No.	Field Name	Type	Constraint	Description
1	option_id	int(11)	PK	Unique option ID
2	plan_id	int(11)	FK → meal_plans(plan_id)	Associated meal plan
3	schedule_type	enum('weekday','extended','full_week')	NOT NULL	Schedule choice (Mon–Fri, Mon–Sat, Mon–Sun)
4	multiplier	decimal(5,2)	DEFAULT 1.00	Price multiplier applied for this schedule
5	description	varchar(255)	NULL	Explanation of the option

21. Table name: popular_meals

Sl. No.	Field Name	Type	Constraint	Description
1	id	int(11)	PK	Record ID
2	meal_id	int(11)	FK → meals(meal_id)	Linked meal
3	popularity_score	int(11)	DEFAULT 0	Score/rank based on orders/feedback
4	created_at	timestamp	DEFAULT current_timestamp()	Added timestamp

22. Table name: subscriptions

Sl. No.	Field Name	Type	Constraint	Description
1	subscription_id	int(11)	PK	Subscription ID
2	user_id	int(11)	FK → users(user_id)	User who subscribed
3	plan_id	int(11)	FK → meal_plans(plan_id)	Subscribed meal plan
4	start_date	date	NOT NULL	Start date
5	end_date	date	NOT NULL	End date
6	status	enum('active', 'paused','cancelled','expired')	DEFAULT 'active'	Subscription state
7	created_at	timestamp	DEFAULT current_timestamp()	Created timestamp
8	updated_at	timestamp	DEFAULT current_timestamp() ON UPDATE	Last updated

CODING

5.CODING

Coding section is where the magic happens. All the planning and the designing done in the previous sections come to life in this section. After this section can only the programmer enjoy the result of his/her hard work when he/she runs the program for the first time.

5.1 SELECTION OF SOFTWARE

PHP

PHP, an acronym for Hypertext Preprocessor, is a versatile server-side scripting language that falls under the broader category of software development. It is widely recognized for its pivotal role in web development and boasts several essential features that make it a preferred choice for building dynamic websites and web applications. Here are some of its key features:

- Open Source
- Database Integration
- Embedded in HTML
- Cross-Platform Compatibility
- Security

MYSQL

MySQL is an open-source relational database system, widely used for web development task like data storage, manipulation, and retrieval. It seamlessly integrates into web applications, eliminating the need for complex setup. MySQL is embedded within web development environments, making administrative tasks effortless. It operates as an SQL-based database, storing data in text files on the device. Unlike systems like JDBC, MySQL simplifies data access with its broad range of relational database features. Its features are

- Zero configuration
- Server less
- Stable cross platform database file
- Less memory
- Self-contained

• Transactional

5.2 CODING PHASE

The goal of the coding or programming phase is to translate the design of the system produced during the design phase into code in a given programming language, which can be executed by a computer and that performs the computation specified by the design. The coding phase affects both testing and maintenance profoundly.

The coding phase does not affect the structure of the system; it has great impact on the internal structure of modules, which affects the testability of the system. The goal of the coding phase is to produce clear simple programs. The aim is not to reduce the coding effect, but to program in a manner so that testing and maintenance costs are reduced.

Programs should not be constructed so that they are easy to write; they should be easy to read and understand. Reading programs is a much more common activity than writing programs. Hence, the goal of the coding phase is to produce simple programs that are clear to understand and modify.

5.2.1 CODING STANDARDS

The standard used in the development of the system is Microsoft Programming standards. It includes naming conversations of variables, constants and objects, standardized formats for labelling and commenting code, spacing, formatting and indenting.

Naming Conventions

In this project, all controls and interface elements are prefixed to clearly indicate their functions and types, ensuring consistency and maintainability throughout the codebase. The following conventions are used:

- **Frames:** Prefixed with frm (e.g., frmMain, frmSettings)
- **Textboxes:** Prefixed with txt (e.g., txtName, txtEmail)
- **Command Buttons:** Prefixed with cmd (e.g., cmdSubmit, cmdCancel)
- **Label Boxes:** Prefixed with lbl (e.g., lblStatus, lblTitle)
- **List Boxes:** Prefixed with lst (e.g., lstOptions, lstUsers)

- **Combo Boxes:** Prefixed with cmb (e.g., cmbPlanType, cmbCity)
- **Date Time Pickers:** Prefixed with dtp or DTP (e.g., dtpStartDate, dtpEndDate)
- **Grid Controls:** Prefixed with grid (e.g., gridDeliveries, gridSubscriptions)

This naming convention is applied across all modules and UI components in the Tiffinly project to facilitate easier identification, code navigation, and future enhancements.

Labels and Comments

The functions of each control are labelled clearly in the GUI. The code also includes comments so that other developers using the source code in future might understand the module functions better.

TESTING

&

IMPLEMENTATION

6. TESTING

Software testing is a critical element of software quality assurance and represents the ultimate review of specifications design and coding. Testing presents an interesting anomaly for the software. Testing is a quality measure process, which reveals the errors in the program. During testing, the program is executed with a set of test cases and the output of the program for the test cases is evaluated to determine if the program is performing as it is expected. Testing plays a very critical role in determining the reliability and efficiency of the software and it is a very important stage in software development.

6.1 TESTING

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based systems. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform allocated functions.

System testing is done in order to ensure that the system developed doesn't fail at any point. Before implementations, the system is tested with experimental data to ensure that it will meets the specified requirements, special tests data are input for processing and results examined.

6.1.1 TEST PLAN

Preparation of test data

Taking various kinds of test data does the testing. Preparation of test data plays a vital role in the system testing. After preparing, the test data the system under study is tested using that test data. While testing the system by using test data errors are again uncovered and corrected by using above testing steps and correction are also noted for future use. Two kinds of test data were collected and used:

- **Using live test data**

Live test is those that are actually extracted from organization files. After a system is partially constructed, programmers or analyst often ask customers to key in a set of data from their normal activities. Then, the system person uses this data as a way to partially test the system.

In order instance, programmers or analysts extract a set of live data from the files and have entered themselves.

- **Using artificial test data**

Artificial test data are created solely for test purpose, since they can be generated to test all combinations of formats and values. In other words, the artificial data, which can quickly be prepared by a data generating utility program in the information system department, make possible the testing of all login and control paths through the program.

The most effective test program uses artificial test data generated by person other than those who wrote the program.

In this project invalid data was entered to test whether the program would break or not. These invalid data entries were randomly generated using random people. Many people were given the software for testing the program. They use gibberish values to test if every validation holds strong.

6.2 TESTING METHODS

Testing is generally done at two levels-testing of individual modules and testing of the entire system. During system testing, the system is used experimentally to ensure that the software does not fail that is, that it will run according to its specifications and the results examined. A limited number of uses may be allowed to use the system so analysis can see whether they use it in unforeseen ways. It is preferable to discover any surprise before the organization implements the system and depends on it.

Testing is done throughout system development at various stages. It is always a good practice to test the system at many different levels at various intervals, that is, sub systems, program modules as work progresses and finally the system as a whole. During testing the major activities are concentrated on the examination ad modification of the source code. Usually, this testing is to be performed by the person other than the person who has really coded it. This is done in order to ensure more complete and unbiased testing for making the software more reliable.

There are two types of testing:

• White box testing

• Black box testing

6.2.1 WHITE BOX TESTING

In white box testing, the internal logic of the modules is considered. Following levels of testing are performed for the developed project:

6.2.1.1 Unit Testing

This involves the tests carried out on modules programs, which make up a system. This is also called as a program testing. The units in a large system many modules at different levels are needed. Unit testing focuses on the modules, independently of one another, to locate errors. The program should be tested for correctness of logic applied and should detect errors in coding. Before proceeding one must make sure that all the programs are working independently.

6.2.2 BLACK BOX TESTING

The concept of the black box is used to represent a system who's inside workings are not available for inspection. In a black box, the test item is treated as "black", since its logic is unknown; all that is known is what goes in and what comes out, or the input and output.

6.2.2.1 System Testing

The system testing is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirement. It falls within scope of black box testing so no knowledge of inner design or logic is needed. As a rule, system testing takes, as its input, all of the integrated software components that have passed integration testing and also the software system itself integrated with any applicable hardware system. The purpose of the integration testing is to detect any inconsistencies between software units.

System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commence. The logical design and the physical design should be thoroughly and continually examined on paper ensure that they will work when implemented.

6.2.2.2 Integration Testing

Integration testing is a systematic technique for constructing the program structure, while at the same time conducting tests to uncover errors associated with interfacing. This is the program is constructed and tested in small segments, which makes it easier to isolate and the following common types of integration problems may be observed:

• Version mistakes
• Data integrity violations
• Overlapping function
• Resource problems especially in memory handling
• Wrong type of parameter in function calls

6.2.2.3 Validation Testing

At the culmination of the integration testing, the software was completely assembled as a package, interfacing errors have been uncovered and corrected and a final series of software validation testing began.

In validation testing we test the system functions in a manner that can be reasonably expected by customer, the system was tested against system requirement specification. Different unusual inputs that the customers may use were assumed and the outputs were verified for such unprecedented inputs. Deviation or errors discovered at this step are corrected prior to the completion of this project with the help of user by negotiating to establish a method for resolving deficiencies.

Thus, the proposed system under consideration has been tested by using validation testing and found to be working satisfactorily. Validation checking is performed on the: -

Numeric Field: Numeric fields (e.g., phone numbers, pin-code, payment amounts) were validated to accept only digits (0-9). Any entry of non-numeric characters triggered an error message. Each module handling numeric input was tested for accuracy and correct error handling.

Character Field: Character fields (e.g., name, city, state) were validated to accept only alphabetic characters (A-Z, a-z). Invalid entries (numbers, special characters) were rejected with appropriate error messages.

Check Null Fields: Before inserting or updating database records, validation was performed to ensure no required fields were left NULL. Attempts to submit forms with missing mandatory data were blocked and users were prompted to complete all fields.

Email Fields: Email fields were validated for correct format and character limits. Invalid email formats and null values were rejected, ensuring only valid email addresses were stored in the database.

Password Fields: Password fields were validated for character limits and required complexity. Invalid or null passwords were rejected, and users were prompted to enter valid passwords.

6.2.3 OUTPUT TESTING

After performing validation test, the next phase is output test of the system, since no system could be useful if it does not produce the desired output in the desired format. By consideration the format of the report/output was generated or displayed and was tested. Here output format was considered in one way: on the display screen.

6.2.4 USER ACCEPTANCE TESTING

User acceptance test of a system is the key factor for the success of the system. The system under consideration was listed for user acceptance by keeping constant touch with the perspective user of the system at the time of design, development and making changes whenever required. This was done with the regards of the following points: -

• Input screen design
• Output design

Users from each of the three roles - **Admin, Customer, and Delivery Partner**—were selected for user acceptance testing. The **Admin** tested the system by logging in and managing meals, subscriptions, and delivery assignments, with all updates correctly stored in the database. The **Customer** tested registration, login, subscription purchase, and payment, ensuring smooth processing and accurate record keeping. The **Delivery Partner** tested login, order acceptance, delivery status updates, and viewing delivery history, with actions properly reflected across the system.

6.3 IMPLEMENTATION

Implementation is the stage of the project when the theoretical design is turned into a working system. The implementation stage is a system project in its own right. It includes careful planning, investigation of current system and its constraints on implementation, design of methods to achieve the changeover, training of the staff in the changeover procedure and evaluation of the changeover method.

The first task in implementation is planning deciding on the methods and time scale to be adopted. Once the planning has been completed the major effort is to ensure that the programs in the system are working properly when the user has been trained.

The complete system involving both computer and user can be executed effectively. Thus, the clear plans are prepared for the activities.

Successful implementation of the new system design is a critical phase in the system life cycle. Implementation means the process of converting a new or a revised system design into an operational one.

MAINTENANCE

&

ENHANCEMENT

7. MAINTENANCE AND ENHANCEMENT

7.1 MAINTENANCE

This software can be modified as need occurs. Maintenance includes all the activities after installation of the software that is performed to keep the system operational. The process of maintenance involves:

➢ Understanding the existing software

➢ Understand the effect of change

➢ Test for satisfaction

This software requires little to no maintenance. During the testing phase most maintenance duties are performed. If a maintenance requirement occurs, it can be solved with ease

7.2 ENHANCEMENT

While Tiffinly already provides a complete digital solution for meal subscription and delivery management, several enhancements can further improve scalability, user experience, and operational efficiency:

1. **Mobile Application Development**
    - Launch dedicated Android/iOS apps for users, admins, and delivery partners.
    - Provide push notifications for order updates, payment confirmations, and delivery status.

2. **AI & Personalization Features**
    - AI-driven meal recommendations based on user preferences and past orders.
    - Nutrition tracking and calorie count integration for health-conscious users.

3. **Enhanced Payment & Wallet System**
    - Integration of a digital wallet within the platform.
    - Loyalty points and reward system for long-term subscribers.

4. **Advanced Analytics Dashboard (Admin)**
    - Real-time sales, subscription trends, popular meal insights, and partner performance.
    - Automated payout calculation for delivery partners.

5. **Route Optimization for Delivery Partners**
    - GPS-based delivery tracking and optimized route suggestions.
    - Reduce delivery delays and fuel costs.

6. **Scalability to Cloud Deployment**
    - Host Tiffinly on cloud platforms (AWS, Azure, GCP) for better performance and reliability.
    - Auto-scaling to handle peak usage during meal times.

7. **Customer Support & Chatbot**
    - AI-powered chatbot for instant support and FAQs.
    - Ticket-based support system for inquiries and complaints.

8. **Subscription Flexibility**
    - Pause, skip, or reschedule deliveries without cancelling entire subscriptions.
    - Trial packs for new users before committing to monthly plans.

9. **Future Expansion**
    - Multi-city or nationwide rollout with region-specific menus.
    - Partnering with local kitchens/home chefs to expand the meal variety.

CONCLUSION

8. CONCLUSION

In today’s fast-paced lifestyle, access to healthy and affordable food is a growing challenge, especially for students and working professionals living away from home. The Tiffinly platform provides a comprehensive digital solution to this problem by integrating meal subscription, delivery management, and feedback mechanisms into a single system.

The project successfully demonstrates how technology can streamline the entire cycle of home-cooked meal services—starting from browsing and customizing meal plans, managing subscriptions, scheduling deliveries, handling secure payments, and ensuring transparency in order tracking. With role-based modules for Users, Admins, and Delivery Partners, the system ensures efficient operation and seamless interaction between all stakeholders.

By incorporating features like CRUD-based management, AJAX-driven interactivity, real-time delivery tracking, and secure session-based authentication, Tiffinly ensures both usability and reliability. Furthermore, the inclusion of a feedback and rating system fosters continuous improvement, enhancing service quality and customer satisfaction.

The platform not only addresses the inefficiencies of the existing manual tiffin system but also introduces scalability for future enhancements, such as AI-driven meal recommendations, mobile app integration, and automated partner payouts. Overall, Tiffinly achieves its goal of digitizing the home-cooked meal subscription ecosystem, making it more accessible, customizable, and transparent.

However, some limitations remain:

- **Limited Real-Time Operations:** Notifications and delivery tracking rely on scheduled updates rather than live GPS-based systems.
- **No Mobile App:** The absence of a dedicated app may limit convenience for both users and delivery partners.
- **Manual Partner Assignment:** Delivery partner allocation and payouts still require administrative involvement.
- **Basic Feedback System:** Lacks advanced analytics and automated responses to negative ratings.

- **Scalability Constraints:** Performance may be affected with a large user base or high delivery volume.
- **Limited Personalization:** AI-driven meal recommendations and dynamic customization are not yet implemented.
- **Payment Integration:** Current payment handling is basic, with limited (dummy razor pay) gateway options and fraud detection.
- **Geographical Restrictions:** Expansion to multiple cities with complex logistics is not fully supported.

By addressing these limitations in future iterations, Tiffinly can evolve into a more robust, scalable, and intelligent platform, further enhancing its value for all stakeholders in the meal subscription ecosystem.

BIBLIOGRAPHY

BIBLIOGRAPHY

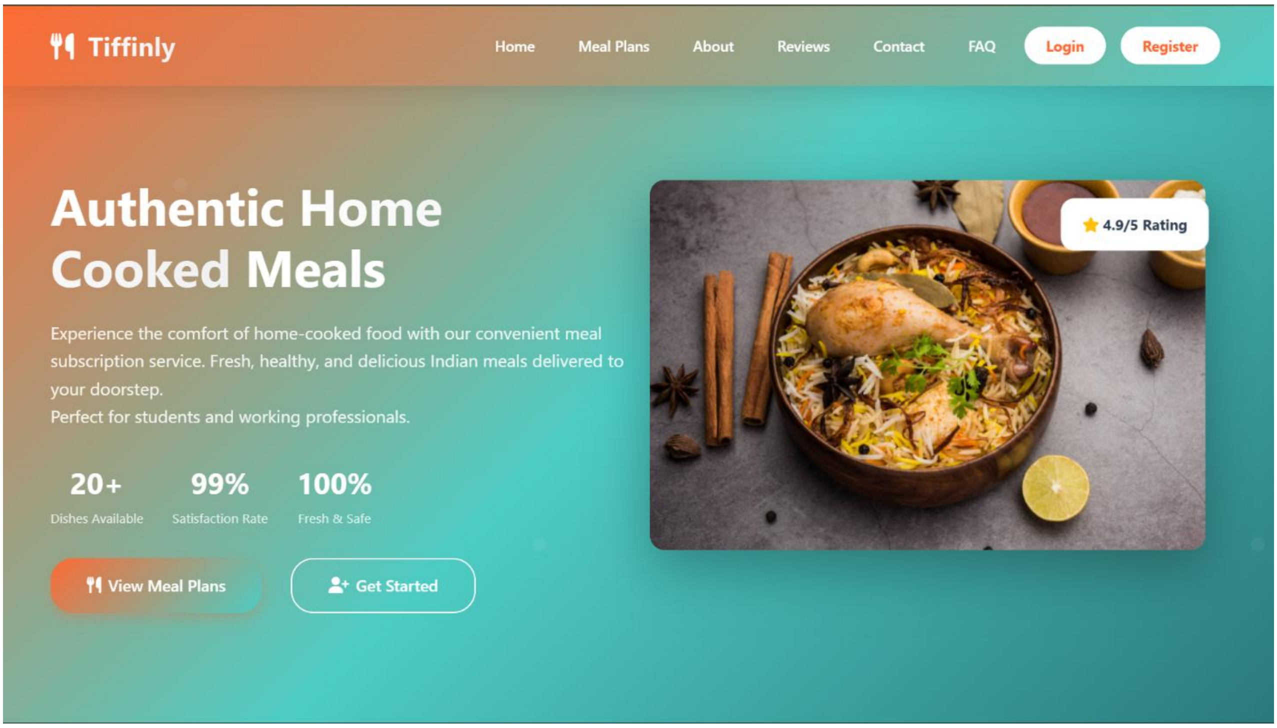
WEBSITES

- www.chatgpt.com
- www.tutorialpoint.com
- www.pinterest.com
- www.pexels.com
- www.w3schools.com

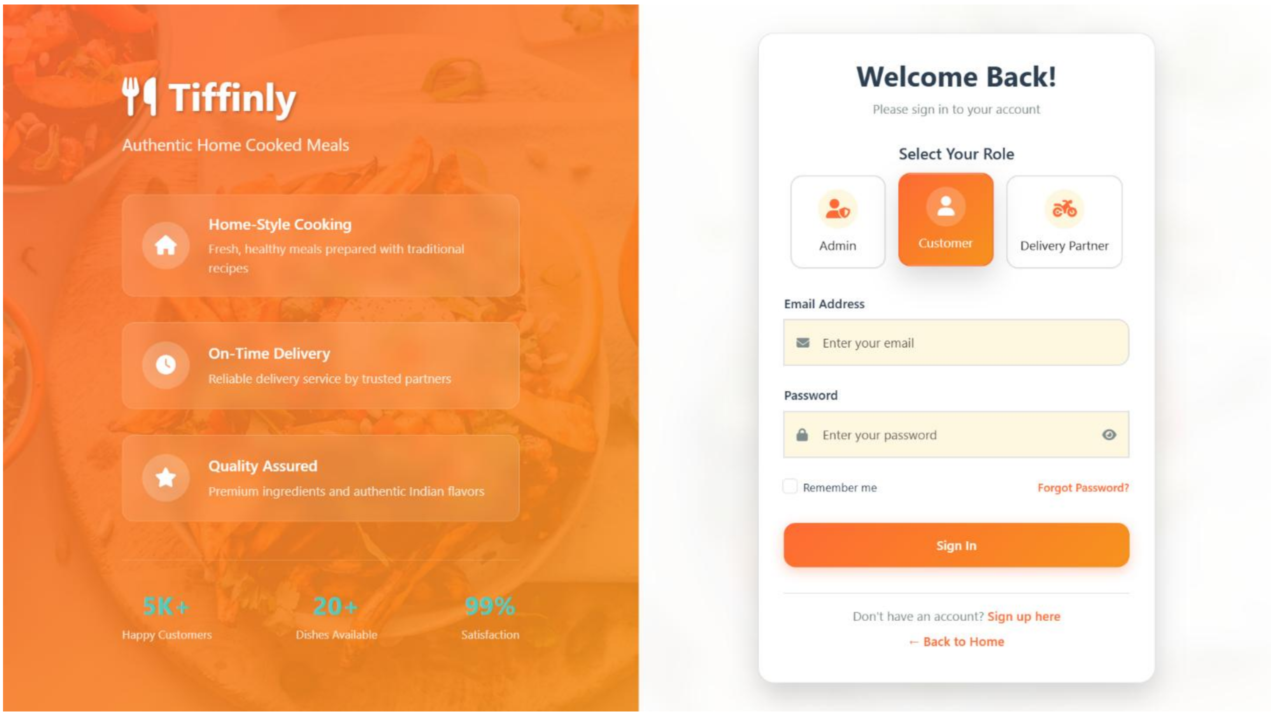
APPENDIX

SCREENSHOTS

1.HOME PAGE

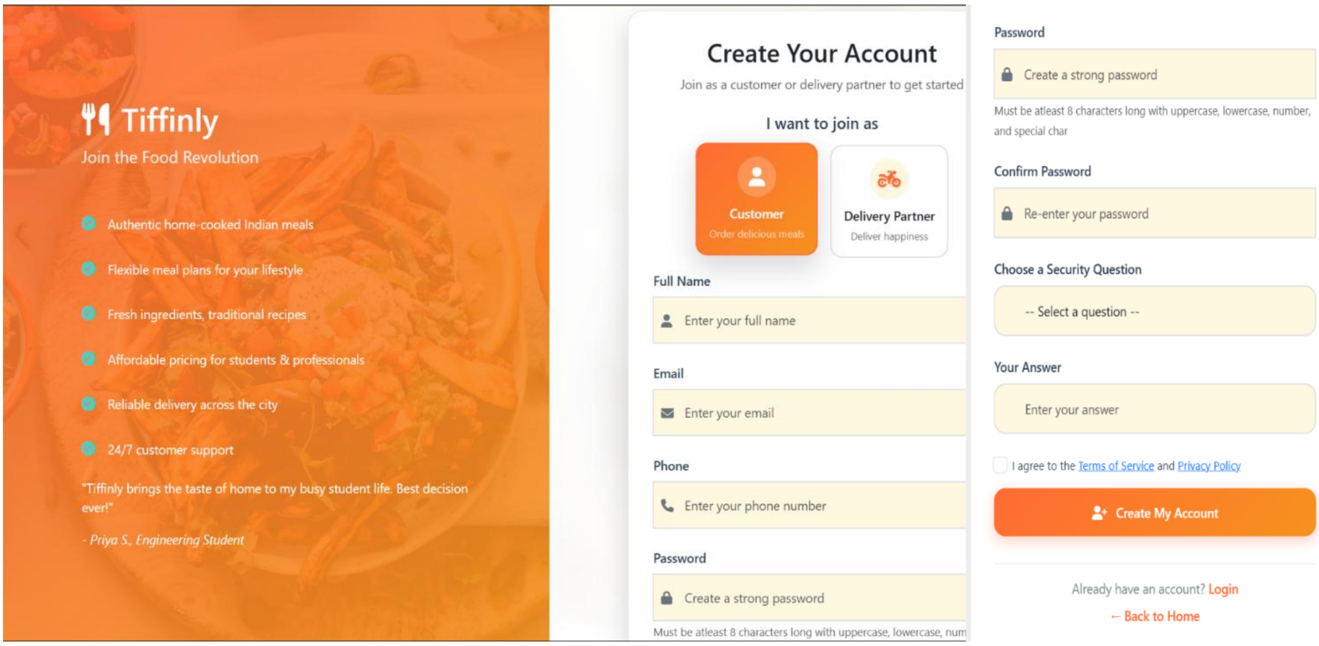


2.LOGIN PAGE

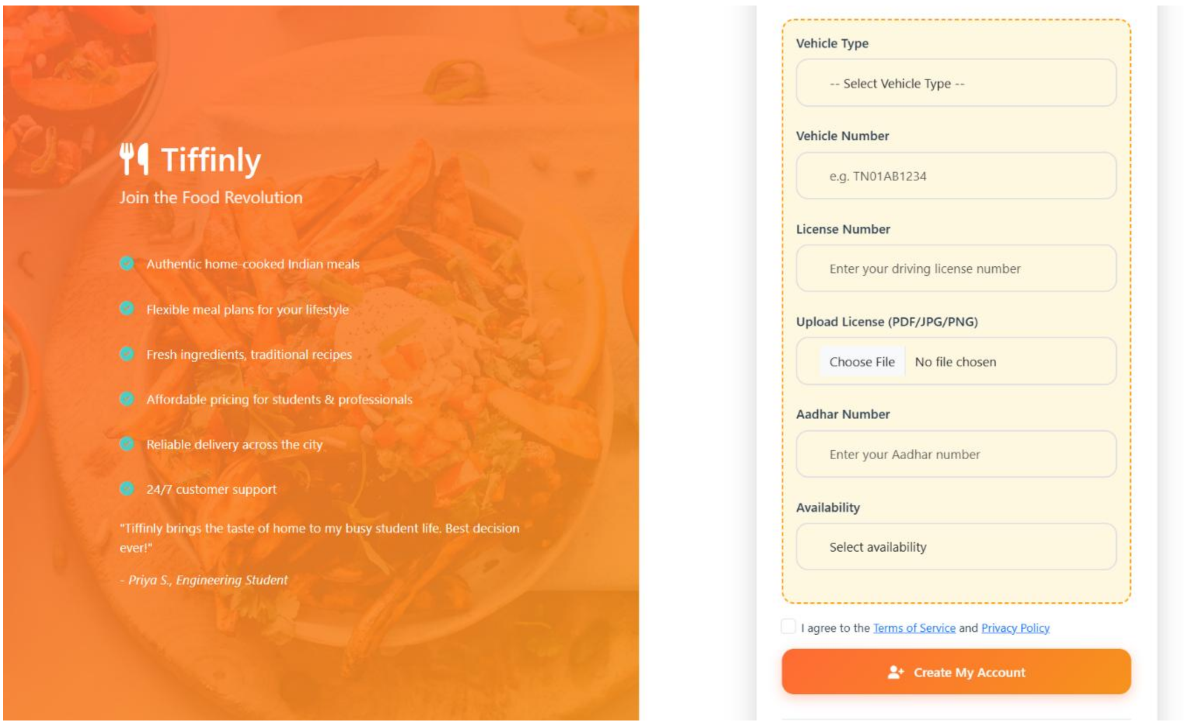


3.REGISTRATION PAGE

A. CUSTOMER REGISTERATION

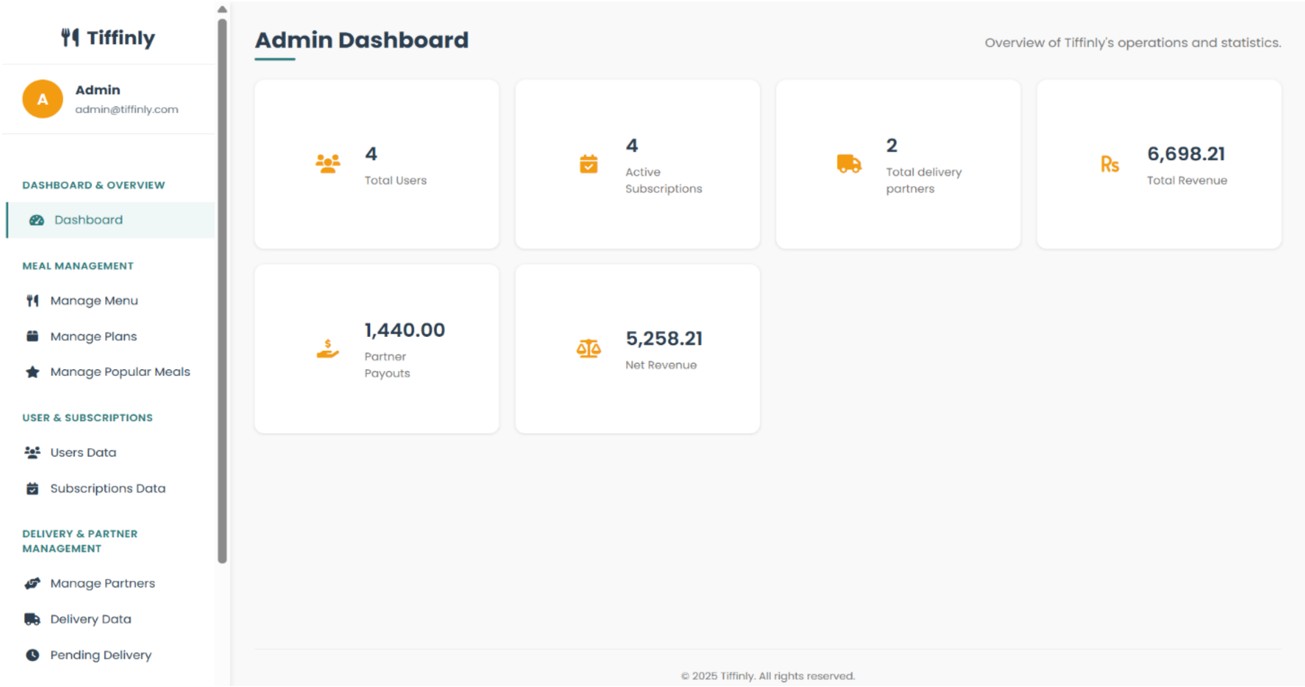


B. DELIVERY PARTNER REGISTERATION (ADDITIONAL FIELDS)

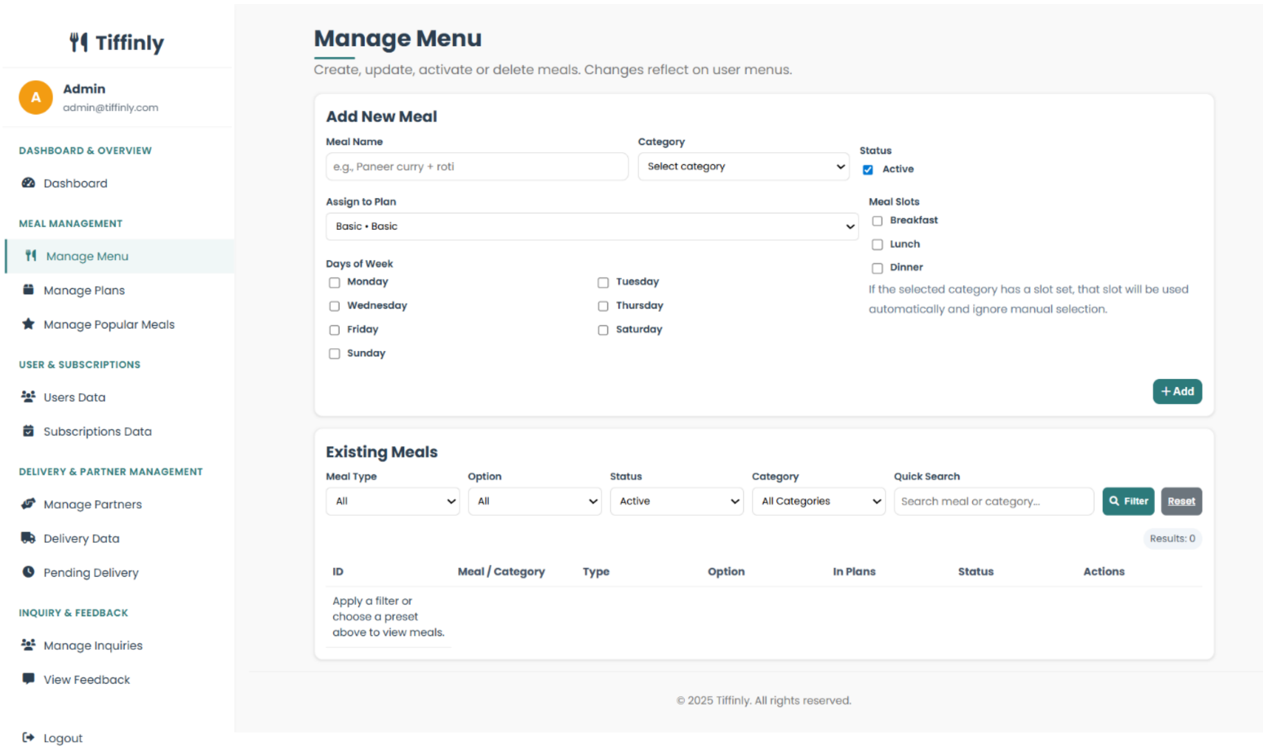


ADMIN MODULE

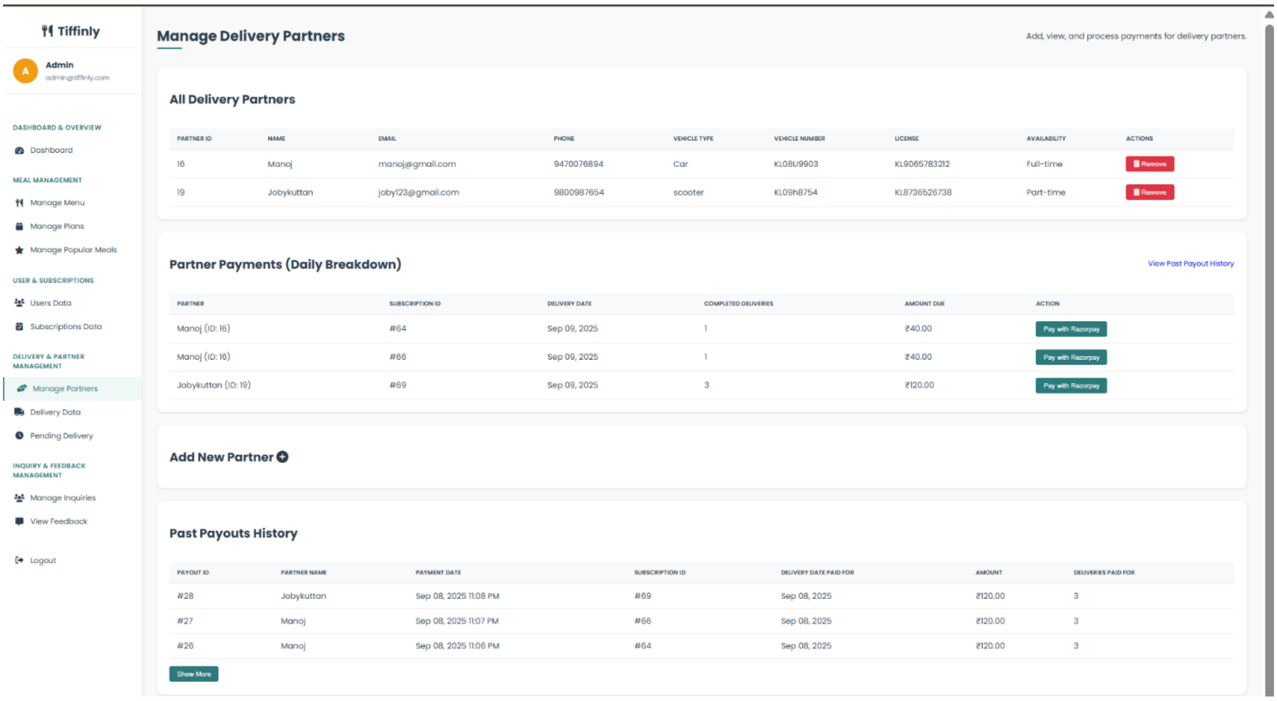
1.ADMIN DASHBOARD



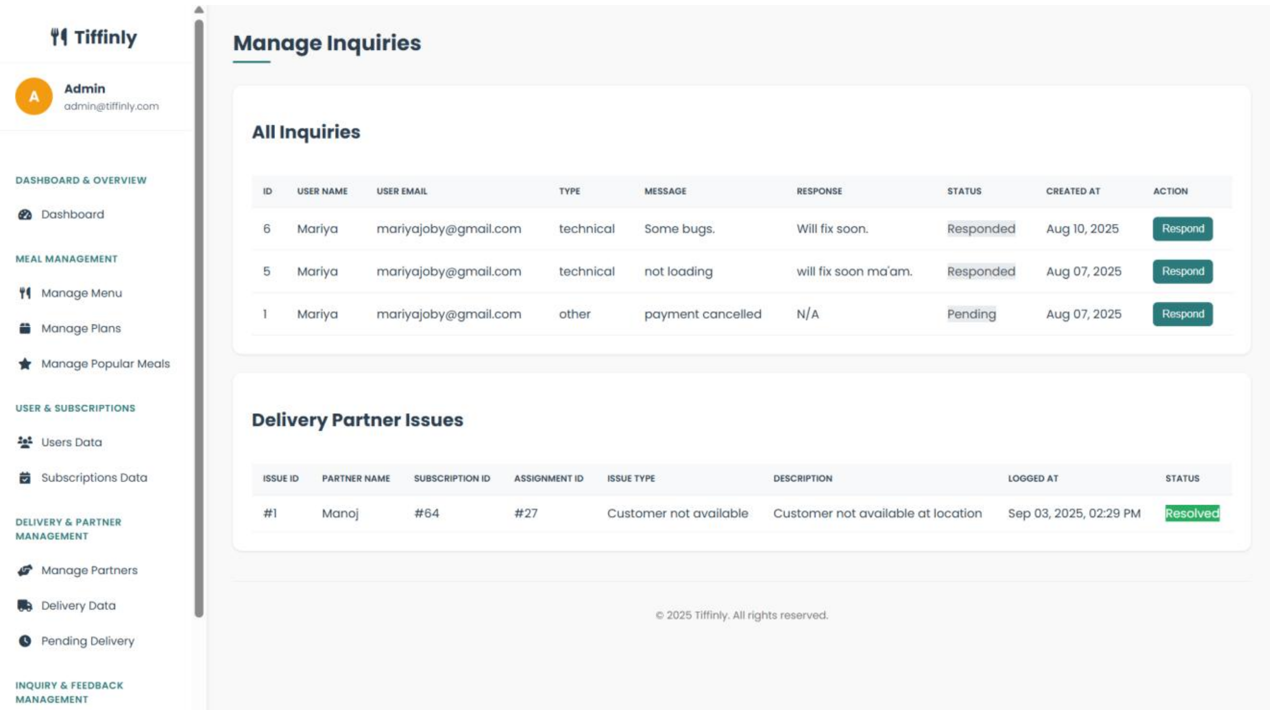
2. MANAGE MENU



3. MANAGE PARTNERS

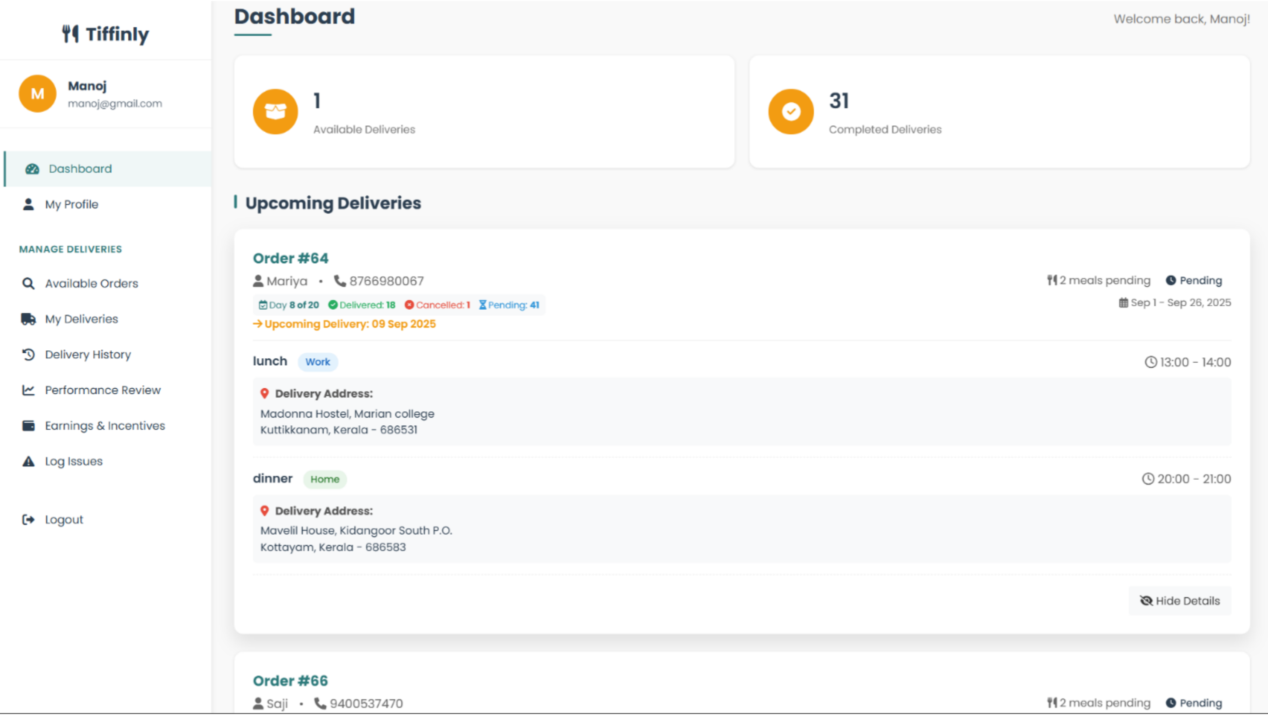


4. MANAGE INQUIRIES

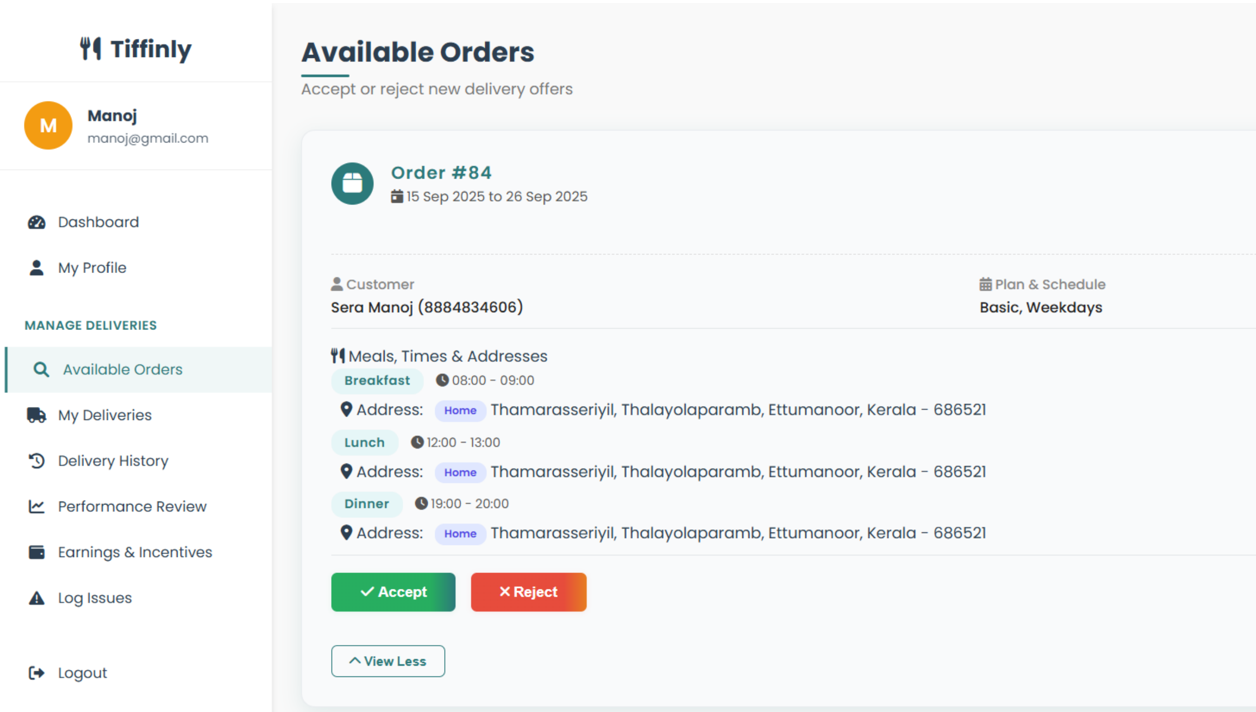


DELIVERY MODULE

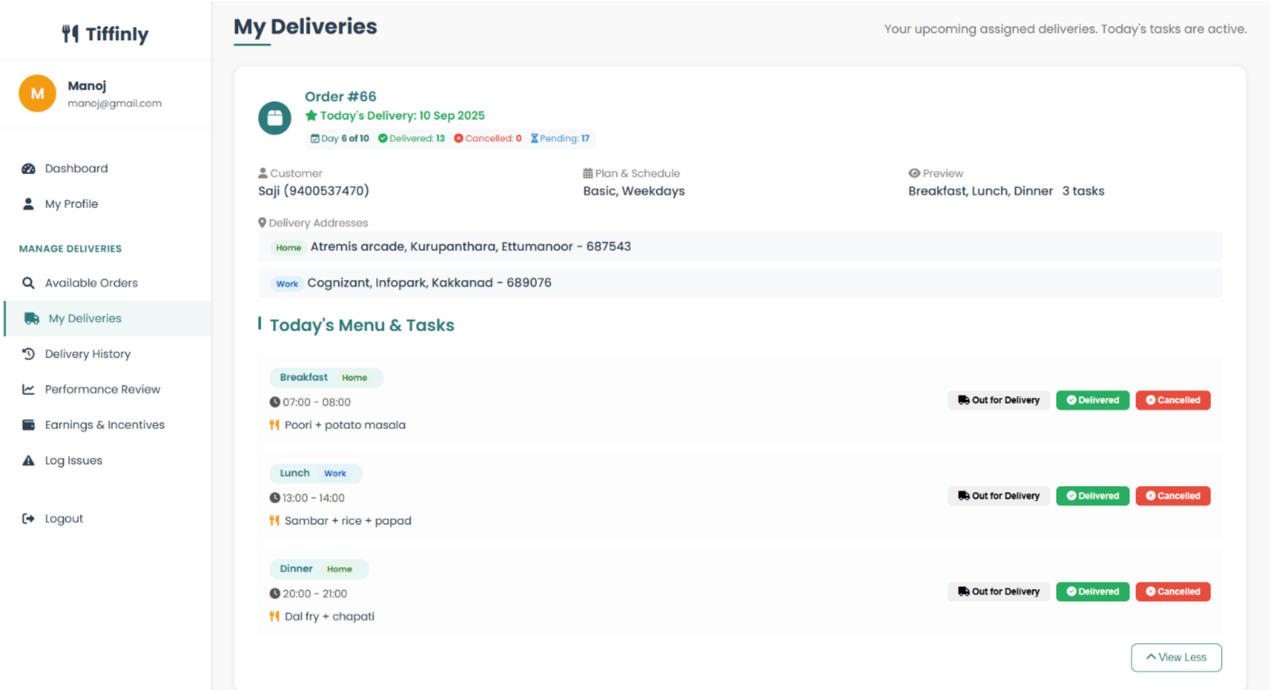
1.PARTNER DASHBOARD



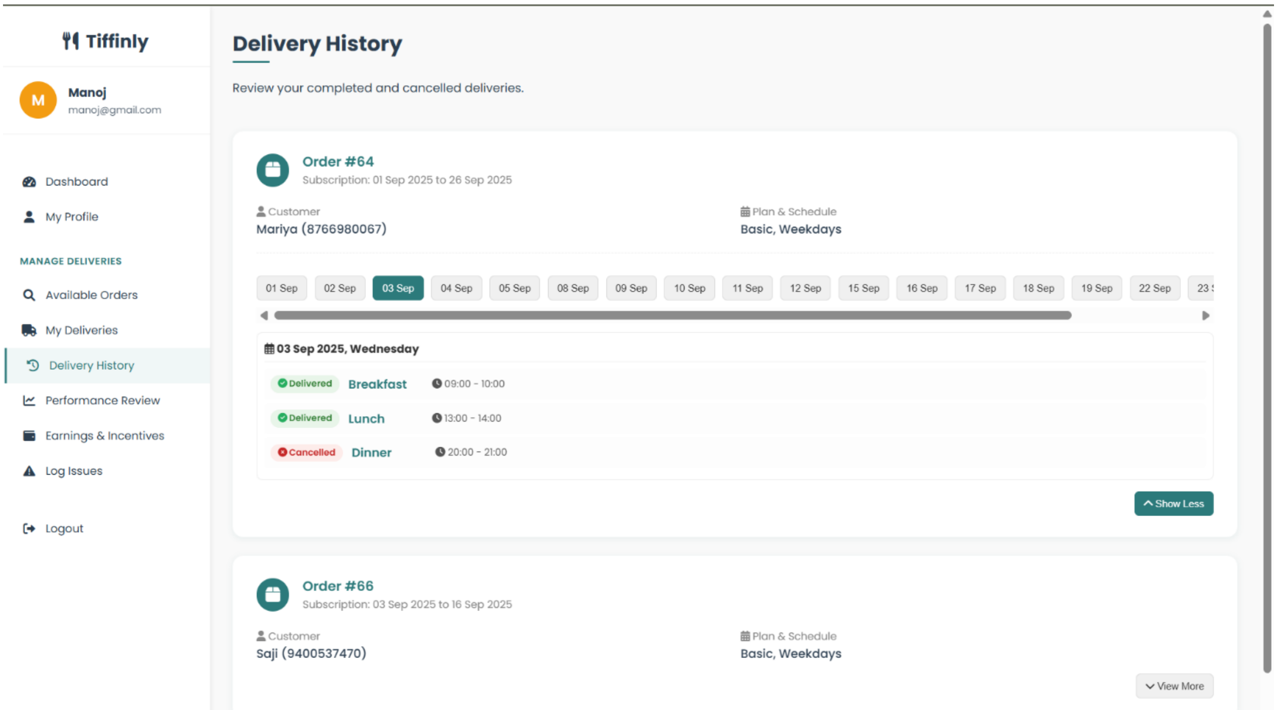
2. AVAILABLE ORDERS



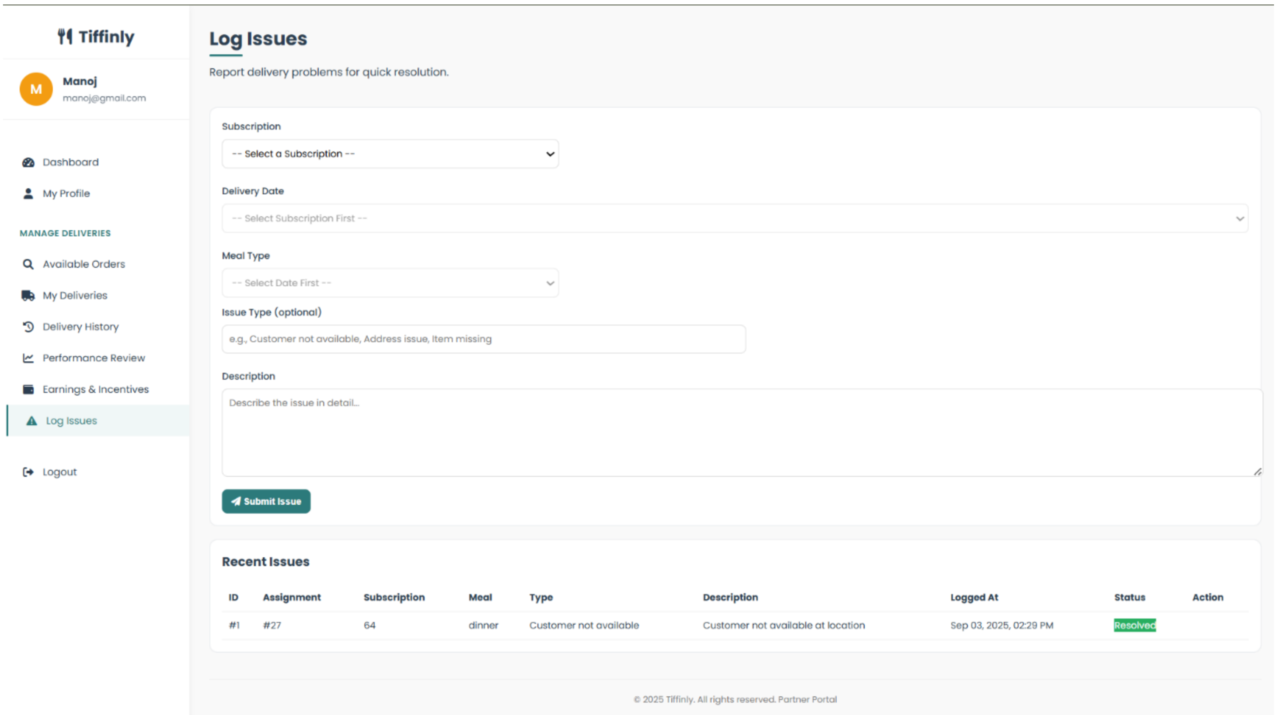
3. MY DELIVERIES



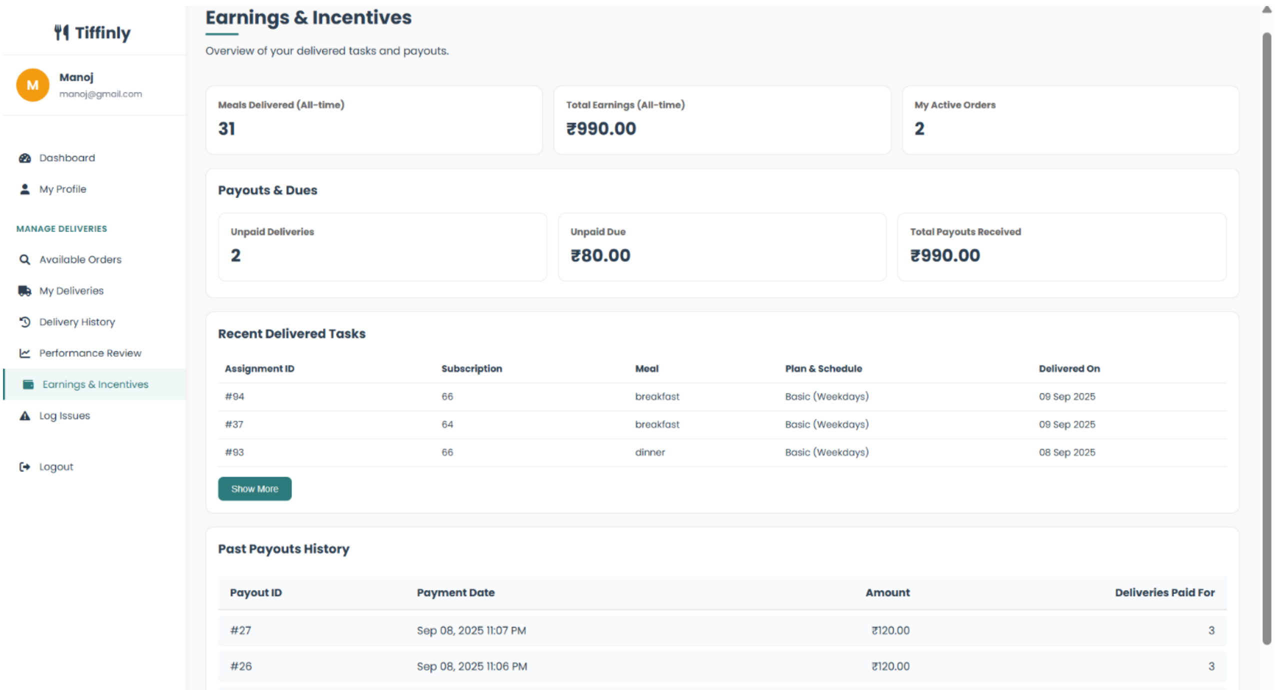
4. DELIVERY HISTORY



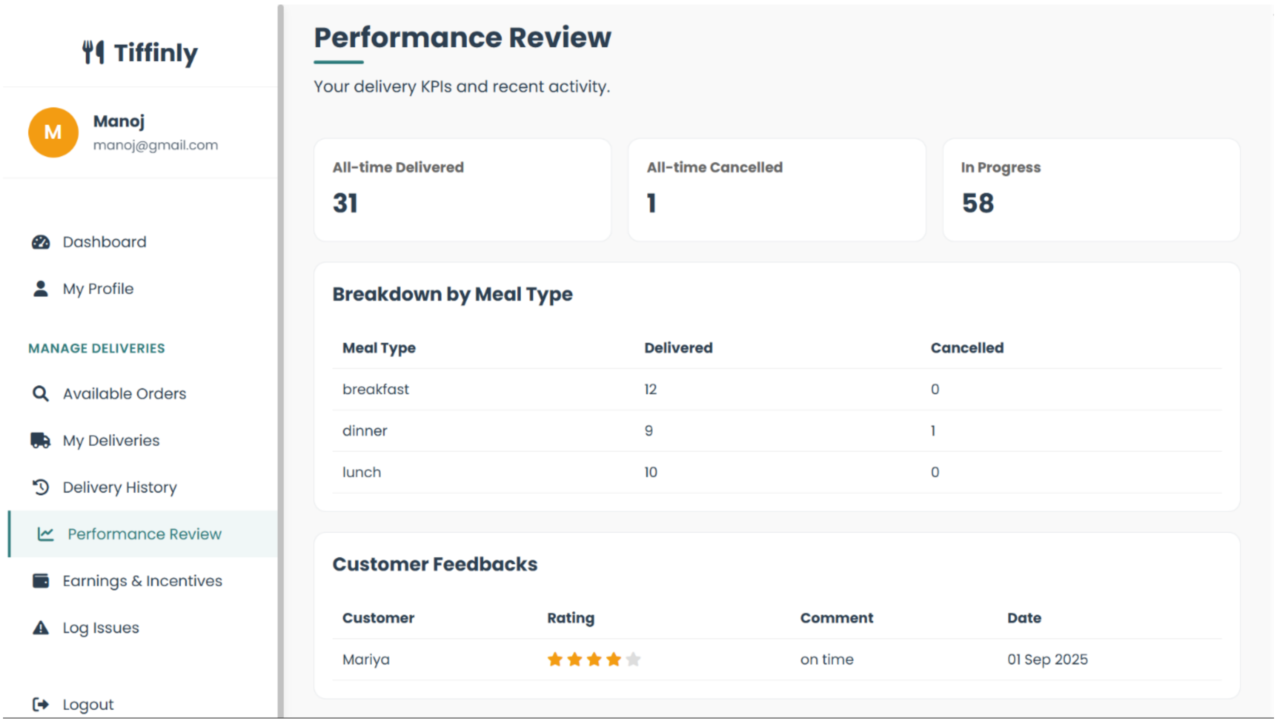
5. LOG ISSUES



6. EARNINGS

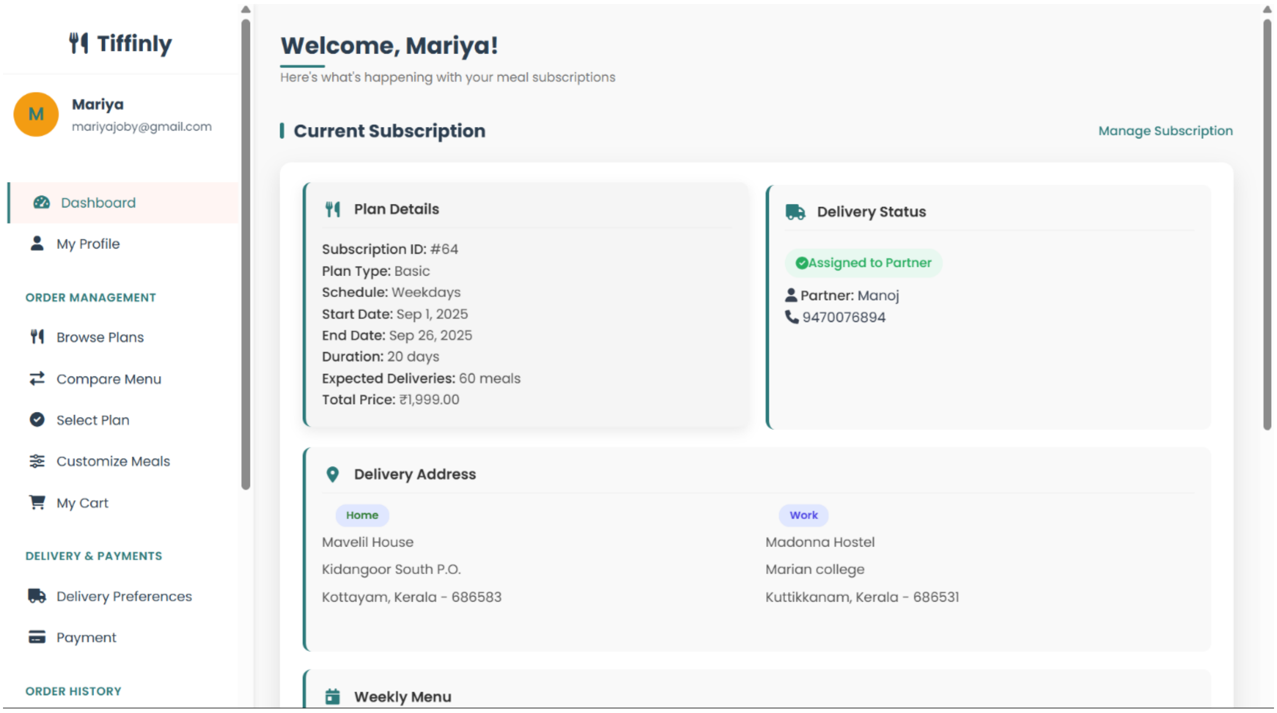


7. PERFORMANCE REVIEW

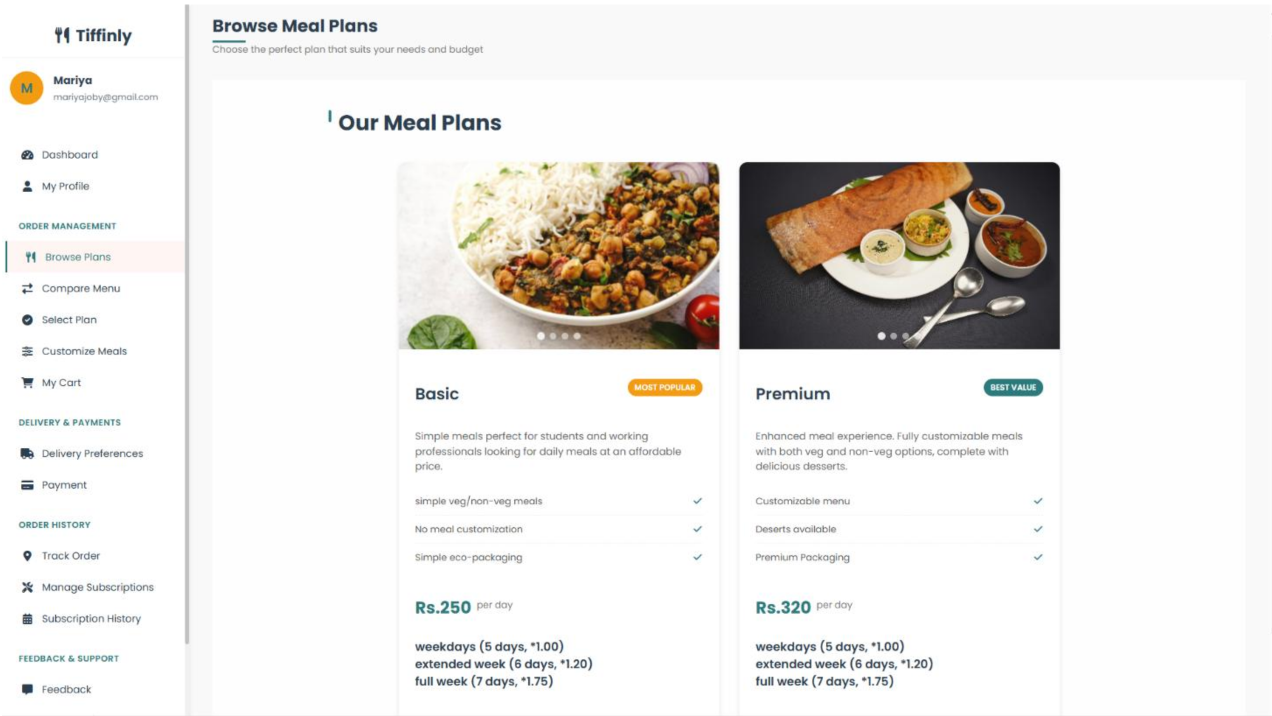


CUSTOMER (USER) MODULE

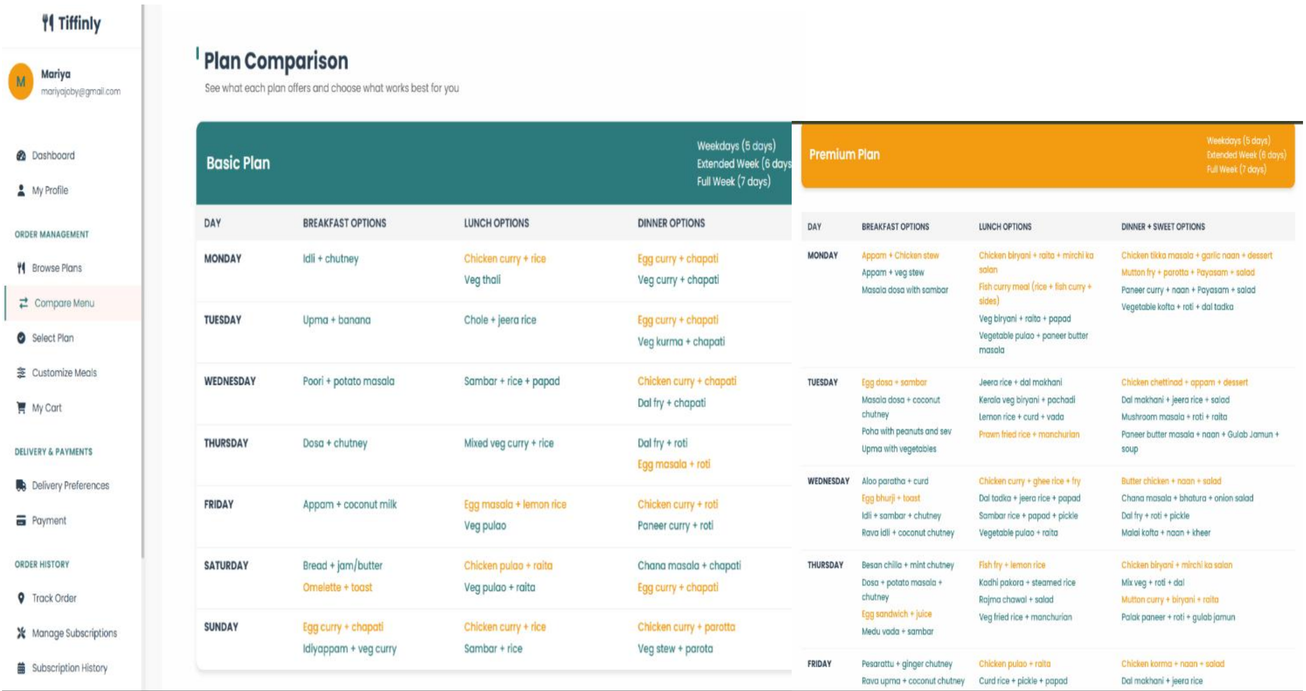
1.CUSTOMER DASHBOARD



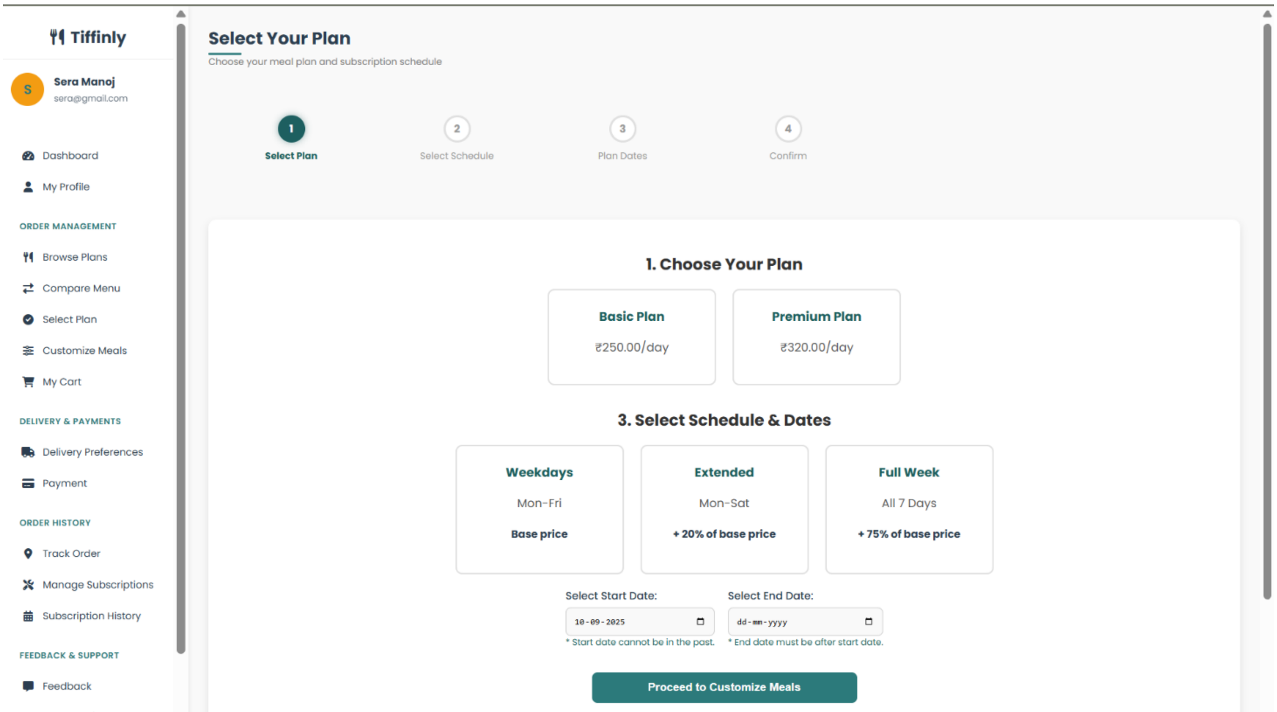
2. BROWSE PLANS



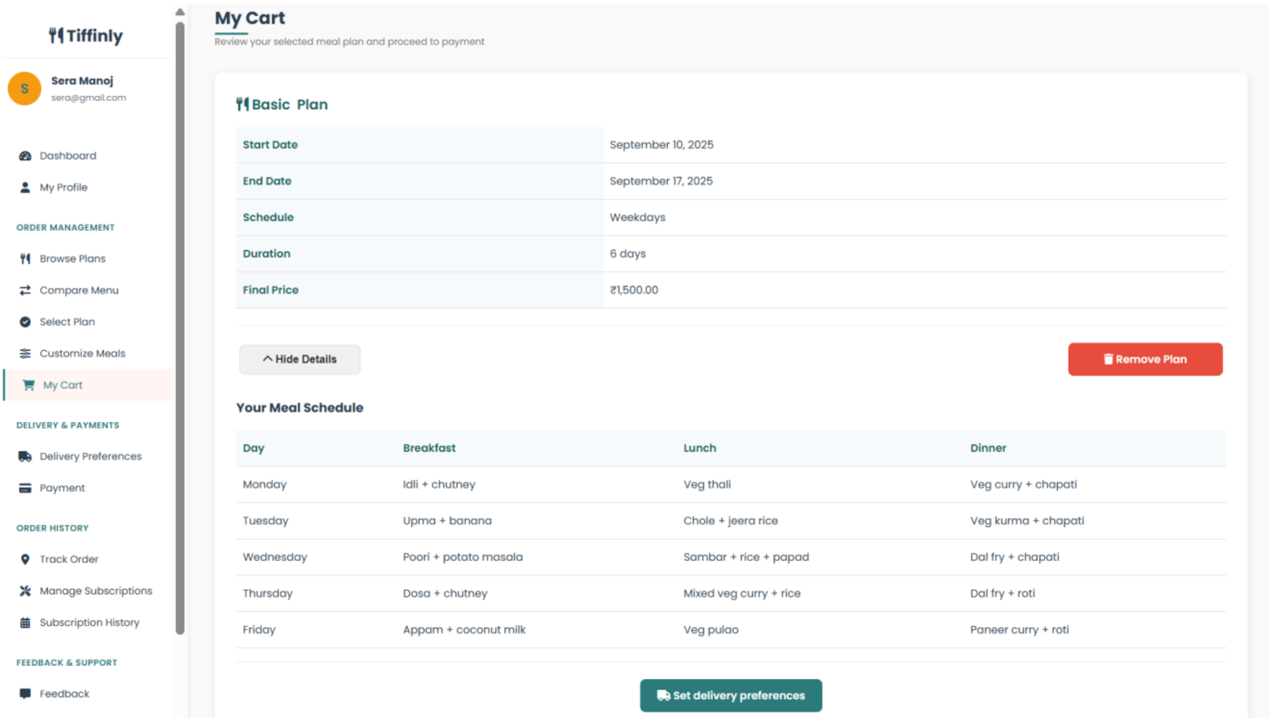
3. COMPARE MENU



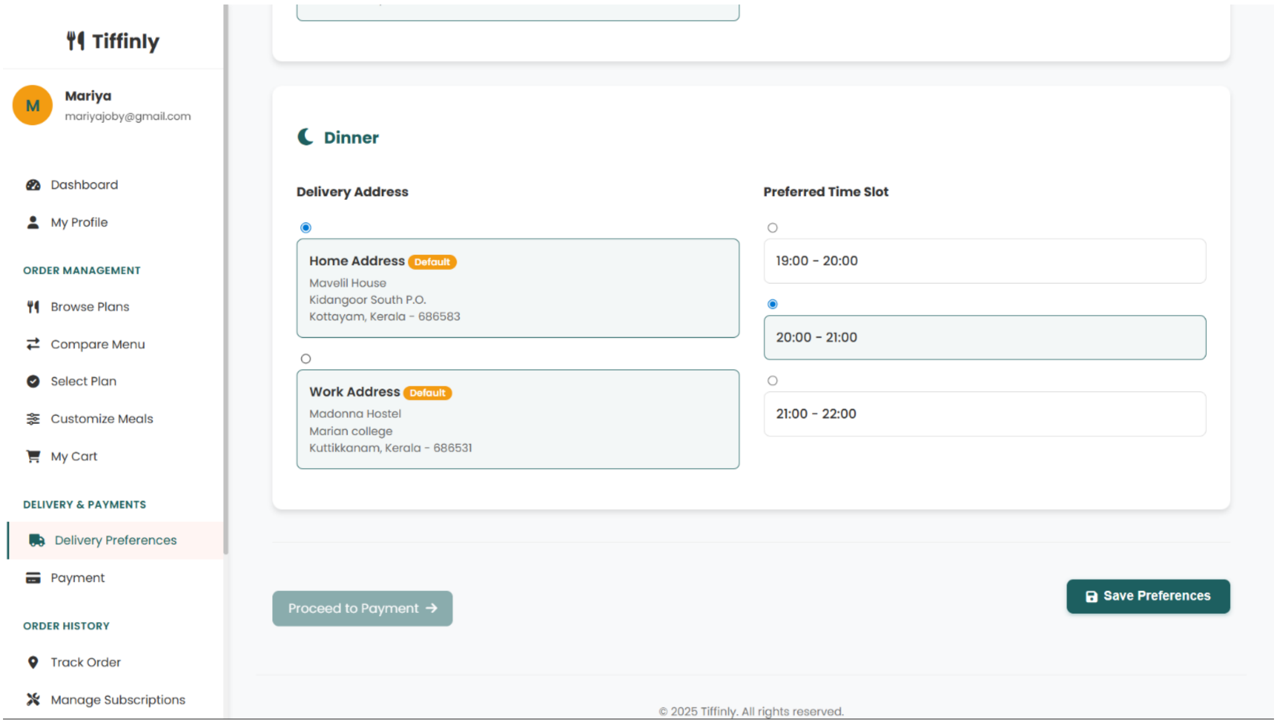
4. SELECT PLAN



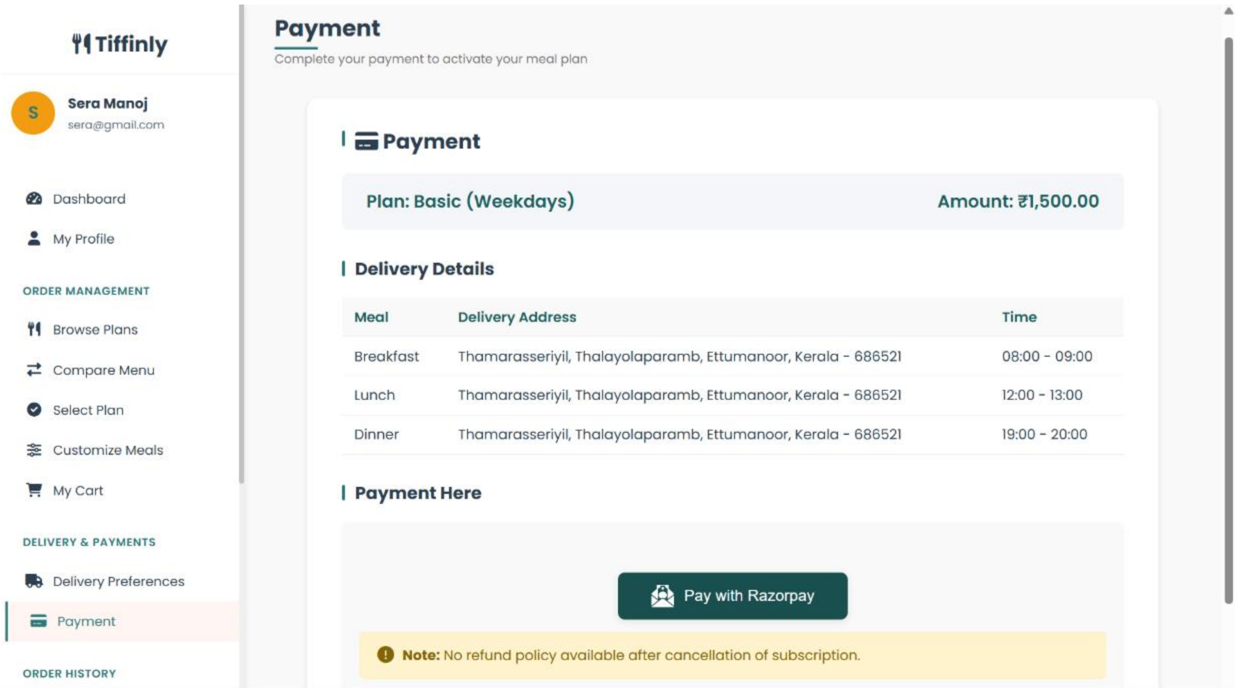
5. CART

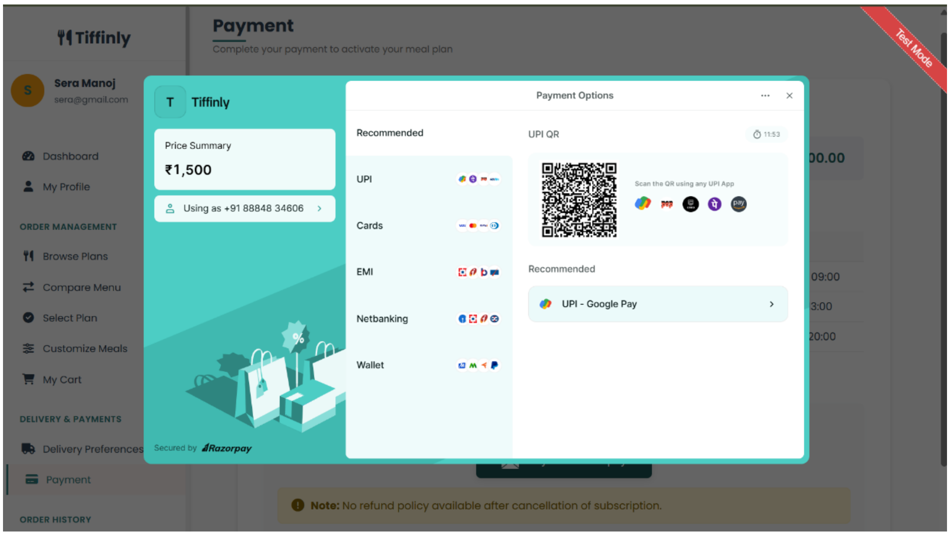


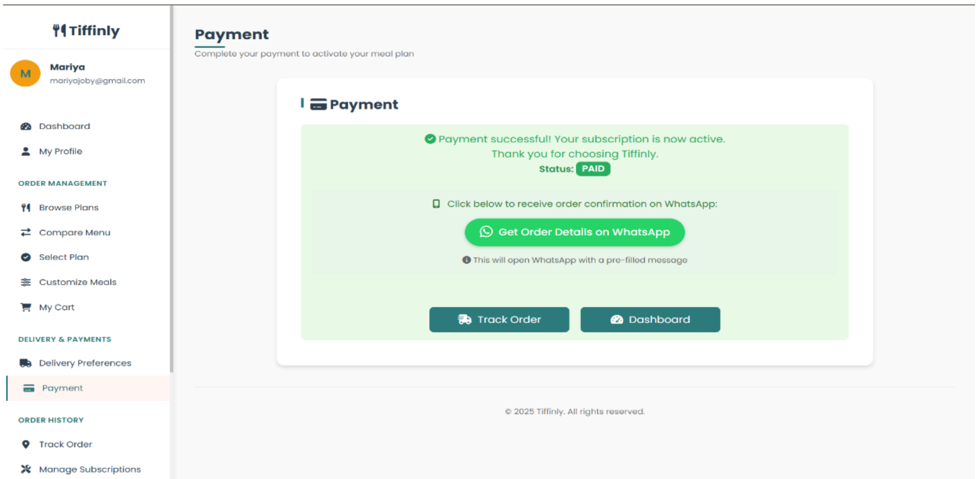
6. DELIVERY PREFERENCES



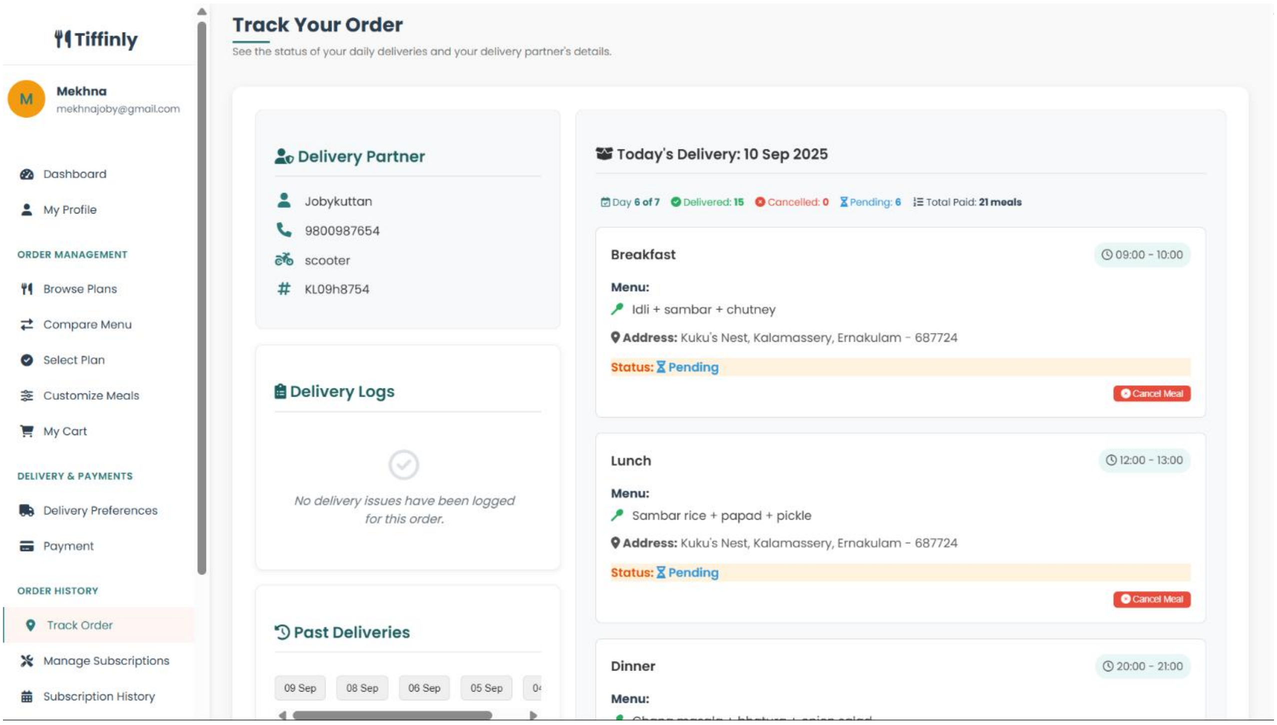
7. PAYMENT



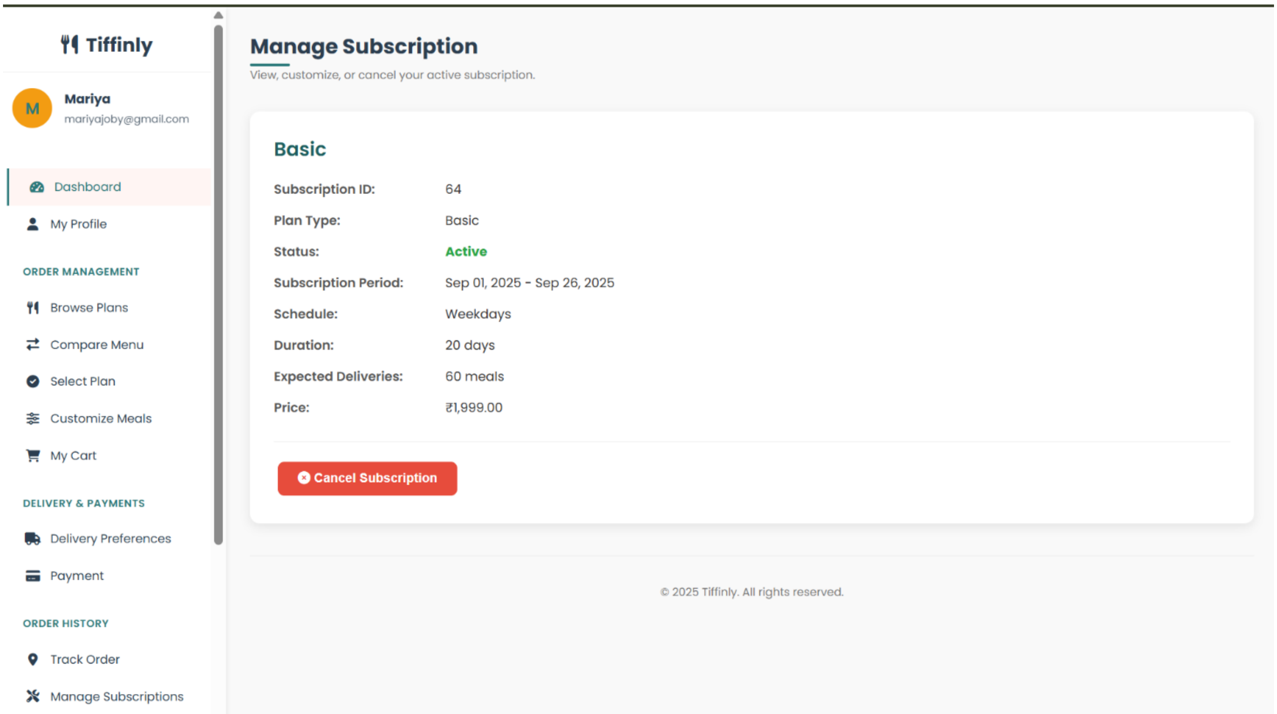




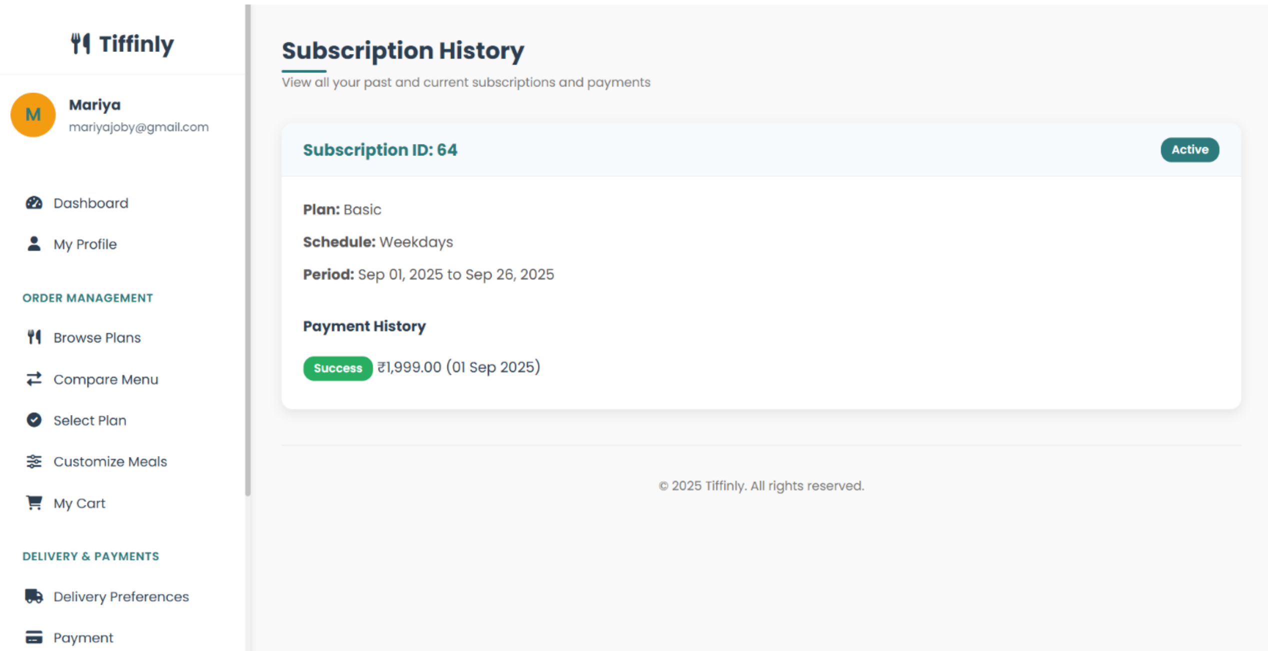
8. TRACK ORDER



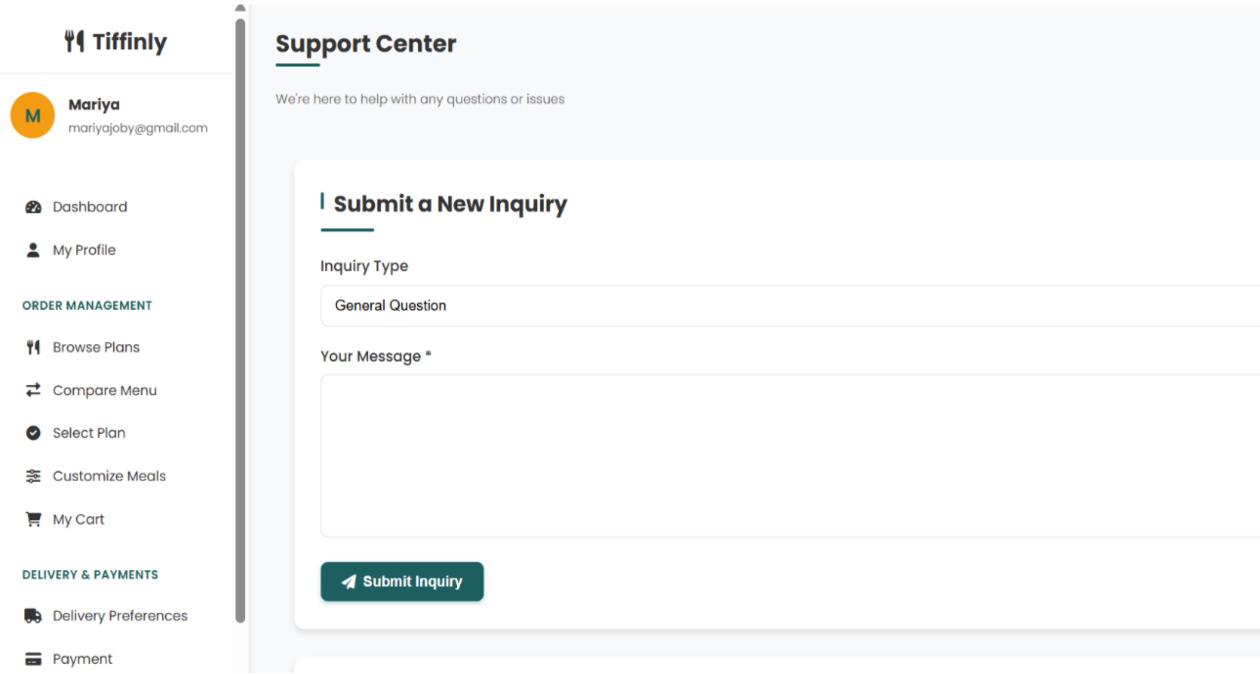
9. MANAGE SUBSCRIPTION



10. SUBSCRIPTION HISTORY



11. SEND INQUIRIES



12. FEEDBACK